

RomWBW

User Guide

Version 3.7

Updated 26 Oct 2026

RetroBrew Computers Group
www.retrobrewcomputers.org

Wayne Warthen
wwarthen@gmail.com

Contents

1 Getting Started **2**
- 1.1 Installation . . . 2
  - 1.1.1 Disk Loaded ROM Images . . . 3
- 1.2 System Startup . . . 3
  - 1.2.1 Core System Information . . . 3
  - 1.2.2 Hardware Discovery . . . 4
  - 1.2.3 Device Unit Assignments . . . 5
- 1.3 Startup Example . . . 6

2 Boot Loader Operation **9**
- 2.1 Starting Applications from ROM . . . 10
- 2.2 Starting Operating Systems from Disk . . . 12
  - 2.2.1 Auto-Submit Batch Files . . . 14
- 2.3 System Management . . . 15
  - 2.3.1 Listing Device Inventory . . . 15
  - 2.3.2 Rebooting the System . . . 16
  - 2.3.3 Setting NVRAM Options . . . 16
  - 2.3.4 Changing Console and Console Speed . . . 17
  - 2.3.5 HBIOS Diagnostic Verbosity . . . 18
- 2.4 Console Takeover . . . 18
- 2.5 Front Panel . . . 19

3 Disk Management **22**
- 3.1 Key Terminology/Concepts . . . 22
- 3.2 Startup Hardware Discovery . . . 23
- 3.3 Drive Letter Assignment . . . 24
  - 3.3.1 Default Drive Letters . . . 25
  - 3.3.2 Assign Drive Letters . . . 26

4 Disk Types **27**
- 4.1 RAM & ROM Disks . . . 27

4.2 Floppy Disks . . . 28
4.3 Hard Disks . . . 29
4.3.1 Hard Disk Layouts . . . 29
4.3.2 Hard Disk Slices . . . 30
4.3.3 Slice Assignment . . . 31
4.3.4 Hard Disk Capacity . . . 32

5 Disk Preparation 33
5.1 Floppy Disk Formatting . . . 33
5.2 Hard Disk Preparation . . . 36
5.2.1 Partition Setup . . . 36
5.2.2 Slice Initialization . . . 38
5.3 Post Disk Preparation . . . 38
5.3.1 Clearing (Formatting) Drives . . . 38
5.3.2 Making a Disk Bootable . . . 39
5.3.3 Copying System Files . . . 40
5.3.4 Checking Disk Layout . . . 40

6 Disk Images 41
6.1 Floppy Disk Images . . . 42
6.2 Hard Disk Images . . . 43
6.2.1 Standard Hard Disk Layout . . . 43
6.2.2 Combo Hard Disk Image . . . 44
6.2.3 Custom Hard Disk Image . . . 45
6.2.4 Writing Hard Disk Images . . . 48
6.2.5 Writing Hard Disk Slices . . . 49

7 Operating Systems 53
7.1 Digital Research CP/M 2.2 . . . 53
7.2 Z-System . . . 56
7.3 NZCOM Automatic Z-System . . . 58
7.4 Digital Research CP/M 3 . . . 59
7.5 Z3PLUS Z-System for CP/M-Plus . . . 61
7.6 ZPM3 . . . 62
7.7 QP/M . . . 65
7.8 UCSD p-System . . . 66
7.9 FreeRTOS . . . 68
7.10 Fuzix . . . 68

8 Custom Applications 71

9 Real Time Clock & Date/Time Stamping 73
9.1 Date/Time Utilities . . . 73
9.1.1 WDATE Utility . . . 74
9.1.2 RTC Utility . . . 74
9.1.3 TESTCLOK Utility . . . 75
9.2 Date/Time File Stamping . . . 76
9.2.1 DateStamper . . . 77
9.2.2 P2DOS (CP/M Plus compatible) . . . 77
9.2.3 NZT . . . 78
9.2.4 Additional Notes . . . 78
9.3 Timezone . . . 79

10 CP/NET Networking 80
10.1 CP/NET Client Setup . . . 81
10.2 CP/NET Sever Setup . . . 83
10.3 CP/NET Usage . . . 84
10.4 Network Boot . . . 86

11 Transferring Files 89
11.1 Serial Port Transfers . . . 89
11.2 Disk Image Transfers . . . 90
11.3 FAT Filesystem Transfers . . . 90
11.3.1 FAT Filesystem Preparation . . . 91
11.3.2 FAT Application Usage . . . 93

12 Customizing RomWBW 95
12.1 Startup Command Processing . . . 95
12.2 NVRAM Configuration . . . 95
12.3 ROM Customization . . . 96
12.4 ROM User Application . . . 96

13 Upgrading 97
13.1 Application Boot . . . 98
13.2 Upgrading via Flash Utility . . . 98
13.3 Upgrading via XModem Flash Updater . . . 99
13.4 Post Upgrade Update . . . 100
13.5 System Update . . . 101

Preface

This document is a general usage guide for the RomWBW software and is generally the best place to start with RomWBW.

On a personal note, I found this document very difficult to write. Members of the retro-computing community have dramatically different experiences, skill levels, and desires. I realize some readers will find this document far too basic. Others will find it lacking in many areas. I am doing my best and encourage you to provide constructive feedback.

Conventions Used

Size Suffixes Within the documentation and in RomWBW in general, the use of size suffixes KB, MB, GB, and TB refer to the binary variant as shown below. The modern suffixes (KiB, MiB, etc.) are not used here because they were not prevalent during the time that the RomWBW OSes were used. This keeps all of RomWBW and associated applications consistent.

Suffix	Value	Meaning
KB	1024	1,024 bytes
MB	1024^2	1,048,576 bytes
GB	1024^3	1,073,741,824 bytes
TB	1024^4	1,099,511,627,776 bytes

Links and URLs Many of the references in the documentation to Internet addresses (URLs) do not provide the address in the text. However, these links are embedded and “clickable” within the documents. Your PDF viewer should highlight these links in some manner (typically an alternate color or an underline).

Chapter 1

Getting Started

1.1 Installation

In general, installation of RomWBW on your platform is very simple. You just need to program your ROM with the correct ROM image from the RomWBW distribution. Subsequently, you can write disk images on your disk drives (IDE disk, CF Card, SD Card, etc.) which then provides even more functionality.

Depending on how you got your hardware, you may have already been provided with a pre-programmed ROM chip. If so, use that initially. Otherwise, you will need to use a ROM programmer to initially program your ROM chip. Please refer to the documentation that came with your ROM programmer for more information.

The fully-built distribution releases are available on the RomWBW Releases Page (https://github.com/wwarthen/RomWBW/releases) of the repository.

The distribution is a .zip archive. After downloading it to a working directory on your modern computer (Windows/Linux/Mac) use any zip tool to extract the contents of the archive.

A complete list of the currently supported platforms is found at the beginning of RomWBW Hardware. You must refer to this document to identify your hardware and the associated ROM Image.

The Binary directory of the distribution contains the pre-built ROM images. There are many files in the Binary directory, but you should be able to locate the ROM Image you identified in RomWBW Hardware. This file which should end in ".rom" must be written to your ROM chip.

Once you have a running RomWBW system, you can generally update your ROM to a newer version in-situ with the included ROM Flashing tool (Will Sowerbutts' FLASH application) as

described in the Upgrading chapter of this document.

1.1.1 Disk Loaded ROM Images

Some hardware platforms do not have sufficient ROM capacity for RomWBW. These platforms have their own ROM which can load the RomWBW ROM Image from disk. The platforms that work this way are annotated in RomWBW Hardware. For these platforms, you will write a special disk image to the boot disk of your system. The disk image is listed in the "ROM Image File" column.

1.2 System Startup

Initially, don't worry about trying to write a disk image to any disk (or CF/SD/USB) devices you have. This will be covered later. You will be able to boot and check out your system with just the ROM.

Connect a serial terminal or computer with terminal emulation software to the primary serial port of your CPU board. You may need to refer to your hardware provider's documentation for details. A null-modem connection may be required. Set the baud rate as indicated in RomWBW Hardware. Set the line characteristics to 8 data bits, 1 stop bit, no parity, and no flow control. If possible, select ANSI or VT-100 terminal emulation. Hardware flow control is not required for terminal operation, but may be necessary for Serial Port Transfers.

RomWBW will automatically attempt to detect and support typical add-on components for each of the systems supported. More information on the required system configuration and optional supported components for each ROM is found in RomWBW Hardware .

Upon power-up, your terminal should display a sign-on banner within 2 seconds followed by hardware inventory and discovery information. When hardware initialization is completed, a boot loader prompt allows you to choose a ROM-based operating system, system monitor, application, or boot from a disk device.

1.2.1 Core System Information

During startup, the first few lines of information displayed provide the most basic information on your system. In the example above, these lines are the Core System Information:

```
RomWBW HBIOS v3.5, 2025-03-01

RCBus [RCZ80_kio] Z80 @ 7.372MHz
0 MEM W/S, 1 I/O W/S, INT MODE 2, Z2 MMU
```

```
512KB ROM, 512KB RAM
ROM VERIFY: 00 00 00 00 PASS
```

The first line is a version identification banner for RomWBW. After that you see a group of 4 lines describing the basic system. In this example, the platform is the RCBus running a configuration named “RCZ80_kio”. The CPU is a Z80 with a current clock speed of 7.372 MHz. There are 0 memory wait states and 1 I/O wait state. Z80 interrupt mode 2 is active and the bank memory manager is type “Z2” which is standard for RCBus. The system has 512KB of ROM total and 512KB of RAM total. Finally, a verification of the checksums of the critical ROM banks is shown (all 4 should be 00).

RomWBW attempts to detect the running configuration of the system at startup. Depending on your hardware, there may be inaccuracies in this section. For example, in some cases the CPU clock speed is assumed rather than actually measured. This does not generally affect the operation of your system. If you want to correct any of the information displayed, you can create a custom ROM which is described later.

1.2.2 Hardware Discovery

The next set of messages during boot show the hardware devices as they are probed and initially configured. In the example above, these lines are:

```
KIO: IO=0x80 ENABLED
CTC: IO=0x84 TIMER MODE=TIM16
AY: MODE=RCZ80 IO=0xD8 NOT PRESENT
SIO0: IO=0x89 SIO MODE=115200,8,N,1
SIO1: IO=0x8B SIO MODE=115200,8,N,1
DSRTC: MODE=STD IO=0xC0 NOT PRESENT
MD: UNITS=2 ROMDISK=384KB RAMDISK=256KB
FD: MODE=RCWDC IO=0x50 NOT PRESENT
IDE: IO=0x10 MODE=RC
IDE0: NO MEDIA
IDE1: NO MEDIA
PPIDE: IO=0x20
PPIDE0: LBA BLOCKS=0x00773800 SIZE=3815MB
PPIDE1: NO MEDIA
```

What you see will depend on your specific system and ROM, but should match the hardware present in your system. Each device has a tag that precedes the colon. This tag identifies the driver and instance of each device. For example, the tag “SIO0:” refers to the SIO serial port driver and specifically the first channel. The “SIO1:” tag refers to the second channel.

In many cases you will see IO=0xNN in the data following the tag. This identifies the base I/O port address of the hardware device and is useful for identifying hardware conflicts.

Note that you may see some lines indicating that the associated hardware is not present. Above, you can see that the FD driver did not find a floppy interface. Lines such as these are completely normal when your system does not have the associated hardware.

Finally, be aware that all ROMs are configured to identify specific hardware devices at specific port addresses. If you add hardware to your system that is not automatically identified, you may need to build a custom ROM to add support for it. Building a custom ROM is covered later.

RomWBW Hardware contains a list of the RomWBW hardware devices which may help you identify the hardware discovered in your system.

1.2.3 Device Unit Assignments

In order to support a wide variety of hardware, RomWBW HBIOS uses a modular approach to implementing device drivers and presenting devices to an operating system. In general, all devices are classified as one of the following:

- Disk (RAM/ROM Disk, Floppy Disk, Hard Disk, CF Card, SD Card, etc.)
- Character (Serial Ports, Parallel Ports, etc.)
- Video (Video Display/Keyboard Interfaces)
- Sound (Audio Playback Devices)
- RTC/NVRAM (Real Time Clock, Non-volatile RAM)
- System (Internal Services, e.g. Timer, DMA, etc.)

HBIOS uses the concept of unit numbers to present a generic set of hardware devices to the operating system. As an example, a typical system might have a ROM Disk, RAM Disk, Floppy Drives, and Disk Drives. All of these are considered disk devices and are presented to the operating system as generic block devices. This means that each operating system does not need to embed code to interact directly with all of the different hardware devices – RomWBW takes care of that.

In the final group of startup messages, a device unit summary table is displayed so that you can see how the actual hardware devices have been mapped to unit numbers during startup.

```
Unit        Device      Type              Capacity/Mode
----------  ----------  ----------------  --------------------
Char 0      UART0:      RS-232            38400,8,N,1
Char 1      UART1:      RS-232            38400,8,N,1
Disk 0      MD1:        RAM Disk          384KB,LBA
```

```
Disk 1      MD0:        ROM Disk          384KB,LBA
Disk 2      FD0:        Floppy Disk       3.5",DS/HD,CHS
Disk 3      FD1:        Floppy Disk       3.5",DS/HD,CHS
Disk 4      IDE0:       CompactFlash      3815MB,LBA
Disk 5      IDE1:       Hard Disk         --
Disk 6      PRPSD0:     SD Card           1886MB,LBA
Video 0     CVDU0:      CRT               Text,80x25
```

In this example, you can see that the system has a total of 7 Disk Units numbered 0-6. There are also 2 Character Units and 1 Video Unit. The table shows the unit numbers assigned to each of the devices. Notice how the unit numbers are assigned sequentially regardless of the specific device.

There may or may not be media in the disk devices listed. For example, the floppy disk devices (Disk Units 2 & 3) may not have a floppy in the drive. Also note that Disk Unit 4 shows a disk capacity, but Disk Unit 5 does not. This is because the PPIDE interface of the system supports up to two drives, but there is only one actual drive attached. A unit number is assigned to all available devices regardless of whether they have actual media installed at boot time.

Note that Character Unit 0 is the initial system console unless modified in a customized ROM image.

If your system has an RTC/NVRAM device, it will not be listed in the unit summary table. Since only a single RTC/NVRAM device can exist in one system, unit numbers are not required nor used for this type of device. Also, System devices are not listed because they are entirely internal to RomWBW.

1.3 Startup Example

Here is an example of a fairly typical startup. Your system will have different devices and configuration, but the startup should look similar.

```
RomWBW HBIOS v3.5, 2025-03-01

RCBus [RCZ80_kio] Z80 @ 7.372MHz
0 MEM W/S, 1 I/O W/S, INT MODE 2, Z2 MMU
512KB ROM, 512KB RAM
ROM VERIFY: 00 00 00 00 PASS

KIO: IO=0x80 ENABLED
CTC: IO=0x84 TIMER MODE=TIM16
AY: MODE=RCZ80 IO=0xD8 NOT PRESENT
```

```
SIO0: IO=0x89 SIO MODE=115200,8,N,1
SIO1: IO=0x8B SIO MODE=115200,8,N,1
DSRTC: MODE=STD IO=0xC0 NOT PRESENT
MD: UNITS=2 ROMDISK=384KB RAMDISK=256KB
FD: MODE=RCWDC IO=0x50 NOT PRESENT
IDE: IO=0x10 MODE=RC
IDE0: NO MEDIA
IDE1: NO MEDIA
PPIDE: IO=0x20
PPIDE0: LBA BLOCKS=0x00773800 SIZE=3815MB
PPIDE1: NO MEDIA

Unit        Device      Type              Capacity/Mode
----------  ----------  ----------------  --------------------
Char 0      SIO0:       RS-232            115200,8,N,1
Char 1      SIO1:       RS-232            115200,8,N,1
Disk 0      MD0:        RAM Disk          256KB,LBA
Disk 1      MD1:        ROM Disk          384KB,LBA
Disk 2      IDE0:       Hard Disk         --
Disk 3      IDE1:       Hard Disk         --
Disk 4      PPIDE0:     CompactFlash      3815MB,LBA
Disk 5      PPIDE1:     Hard Disk         --
```

If your system completes the ROM-based boot process successfully, you should see the RomWBW Boot Loader prompt. For example:

```
RCBus [RCZ80_kio] Boot Loader


Boot [H=Help]:
```

If you get to this prompt, your system has completed the boot process and is ready to accept commands. Note that the Boot Loader is not an operating system or application. It is essentially the point where you choose which operating system or application you want RomWBW to execute.

The Boot Loader is explained in detail in the next section. For now, you can try a few simple commands to confirm that you can interact with the system.

At the Boot Loader prompt, you can type `H <enter>` for help. You can type `L <enter>` to list the available built-in ROM applications. If your terminal supports ANSI escape sequences, you can try the 'P' command to play a simple on-screen game. Instructions for the game are found in RomWBW Applications.

If all of this seems fine, your ROM has been successfully programmed.

Chapter 2

Boot Loader Operation

Once your system has completed the startup process, it presents a Boot Loader command prompt. The purpose of the Boot Loader is to select and launch a desired application or operating system. It also has the ability to configure some aspects of system operation.

After starting your system, following the hardware initialization, you will see the RomWBW Boot Loader prompt. Below is an example. Note that the text preceding “Boot Loader” will vary and identifies your specific system and configuration.

```
Mark IV [MK4_wbw] Boot Loader

Boot [H=Help]:
```

From the Boot Loader prompt, you can enter commands to select and launch any of the RomWBW operating systems or ROM applications. It also allows you to manage some basic settings of the system. To enter a command, just enter the command followed by ***<enter>***.

For example, typing `H<enter>` will display a short command summary:

```
Boot [H=Help]: H

  L            - List ROM Applications
  D            - Device Inventory
  S            - Slice Inventory
  R            - Reboot System
  W            - RomWBW Configure
  I <u> [<c>] - Set Console Interface/Baud Rate
  V [<n>]      - View/Set HBIOS Diagnostic Verbosity
  N            - Network Boot
```

```
  <u>[.<s>]   - Boot Disk Unit/Slice
```

Likewise the L command (List ROM Applications) will display the list of ROM Applications that you can launch right from the Boot Loader:

```
Boot [H=Help]: L

ROM Applications:

  M: Monitor
  Z: Z-System
  C: CP/M 2.2
  F: Forth
  B: BASIC
  T: Tasty BASIC
  P: Play a Game
  X: XModem Flash Updater
  U: User App
```

A more complete description of these options is found below in System Management.

2.1 Starting Applications from ROM

To start a ROM application you just enter the corresponding letter at the Boot Loader prompt. In the following example, we launch the built-in Microsoft BASIC interpreter. From within BASIC, we use the BYE command to return to the Boot Loader:

```
Boot [H=Help]: b

Loading BASIC...
Memory top?
Z80 BASIC Ver 4.7b
Copyright (C) 1978 by Microsoft
55603 Bytes free
Ok
bye


Mark IV [MK4_wbw] Boot Loader

Boot [H=Help]:
```

The following ROM applications and OSes are available at the boot loader prompt:

Application	Description
Monitor	Z80 system debug monitor w/ Intel Hex loader
CP/M 2.2	Digital Research CP/M 2.2 OS
Z-System	ZSDOS 1.1 w/ ZCPR 1 (Enhanced CP/M compatible OS)
Forth	Brad Rodriguez's ANSI compatible Forth language
BASIC	Microsoft ROM BASIC
Tasty BASIC	Dimitri Theuling's Tiny BASIC implementation
Play	A simple video game (requires ANSI terminal emulation)
Flash Update	Upload and flash a new ROMWBW image using xmodem
User App	User written application placeholder

The User App is provided as a way to access a custom written ROM module. In the pre-built ROMs, selecting User App will just return to the Boot Loader menu. If you are interested in creating a custom application to run here, review the "usrrom.asm" file in the Source/HBIOS folder of the distribution.

Each of the ROM Applications is documented in RomWBW Applications. Some of the applications (such as BASIC) also have their own independent manual in the Doc directory of the distribution. The OSes included in the ROM (CP/M 2.2 & Z-System) are described in the Operating Systems chapter of this document.

In general, the command to exit any of these applications and restart the system is `BYE`. The exceptions are the Monitor which uses `X` and Play which uses `Q`.

NOTE: Of the ROM Applications, only the operating systems (CP/M and Z-System) have the ability to interact with disk drives. So, other than these 2 OSes, the ROM Applications do **not** have any way to save or load data from persistent/disk storage. For example, if you launch BASIC from the Boot Loader, you will not be able to save or load your programs. You will need to start an operating system first and then run BASIC in order to save or load programs.

Two of the ROM Applications are, in fact, complete operating systems. Specifically, "CP/M 2.2" and "Z-System" are provided so that you can actually start either operating system directly from your ROM. This technique is useful when:

- You don't yet have any real disk drives in your system
- You want to setup real disk drives for the first time
- You are upgrading your system and need to upgrade your real disk drives

The RAM disk and ROM disk drives will be available even if you have no physical disk devices attached to your system.

Booting an operating system from ROM is not intended as a way to use your operating system on a long-term basis. The ROM disk has only a small subset of the operating system files. Additionally, you cannot easily customize your ROM disk because you cannot write to it. For any significant use of an operating system, you should boot directly to the disk/slice that contains the complete operating system. This is described in the next section.

2.2 Starting Operating Systems from Disk

In order to make use of the more sophisticated operating systems available with RomWBW, you will need to boot an operating system from a disk. Setting up disks is described in detail later. For now, we will just go over the command line for performing this type of boot.

From the Boot Loader prompt, you can enter a number (***<diskunit>***) and optionally a dot followed by a second number (***<slice>***). The ***<disk unit>*** unit number refers to a disk unit that was displayed when the system was booted – essentially it specifies the specific physical disk drive you want to boot. The ***<slice>*** numbers refers to a portion of the disk unit to boot. If no slice is specified, then it is equivalent to booting from the first slice (slice 0). Disk units and slices are described in more detail later.

Following this, you should see the operating system startup messages. Your operating system prompt will typically be `A>` and when you look at the drive letter assignments, you should see that A: has been assigned to the disk and slice you selected to boot.

If you receive the error message “Disk not bootable!”, you have either failed to properly initialize the disk and slice requested or you have selected an invalid/unavailable disk/slice.

The following example shows a disk boot into the first slice of disk unit 4 which happens to be the CP/M 2.2 operating system on this disk. This is accomplished by entering just the number ‘4’ and pressing ***<enter>***.

```
Boot [H=Help]: 4

Booting Disk Unit 4, Slice 0, Sector 0x00000800...

Volume "Unlabelled" [0xD000-0xFE00, entry @ 0xE600]...

CBIOS v3.1.1-pre.194 [WBW]

Formatting RAMDISK...

Configuring Drives...
```

```
        A:=IDE0:0
        B:=MD0:0
        C:=MD1:0
        D:=FD0:0
        E:=FD1:0
        F:=IDE0:1
        G:=IDE0:2
        H:=IDE0:3
        I:=PRPSD0:0
        J:=PRPSD0:1
        K:=PRPSD0:2
        L:=PRPSD0:3

        1081 Disk Buffer Bytes Free

CP/M-80 v2.2, 54.0K TPA

A>
```

Notice that a list of drive letters and their assignments to RomWBW devices and slices is displayed during the initialization of the operating system.

Here is another example where we are booting disk unit 4, slice 3 which is the CP/M 3 operating system on this disk:

```
Boot [H=Help]: 4.3

Booting Disk Unit 4, Slice 3, Sector 0x0000C800...

Volume "Unlabelled" [0x0100-0x1000, entry @ 0x0100]...

CP/M V3.0 Loader
Copyright (C) 1998, Caldera Inc.

 BNKBIOS3 SPR  F600  0800
 BNKBIOS3 SPR  4500  3B00
 RESBDOS3 SPR  F000  0600
 BNKBDOS3 SPR  1700  2E00

 60K TPA
```

```
CP/M v3.0 [BANKED] for HBIOS v3.5

A>
```

Some operating systems (such as CP/M 3 shown above) do not list the drive assignments during initialization. In this case, you can use the `ASSIGN` command to display the current assignments.

The Boot Loader simply launches whatever is in the disk unit/slice you have specified. It does not know what operating system is at that location. The layout of operating systems on disk media is described in the Disk Images section of this document.

2.2.1 Auto-Submit Batch Files

All of the operating systems supplied with RomWBW have the ability to execute a "batch" of commands by creating a batch submission file containing the commands to be executed. The mechanism for running commands automatically at startup varies by operating system. In some cases, it was built into the original operating system. In other cases, I have added this capability in the RomWBW BIOS of the operating system.

In all cases, the file containing the commands to run at startup must be on the boot drive (A:). RomWBW automatically assigns A: to the disk slice you choose to boot. Adding a startup command file to the ROM Disk is not recommended because it would require customizing and building a new ROM. Use of bootable disk slices is preferred since the startup command files can be added/edited without any special system customization.

Here is an overview for each operating system:

- **CP/M 2.2** - Will run PROFILE.SUB as a SUBMIT file if it exists in A: at startup. Note that original CP/M 2.2 itself did not have this ability – it was added to the RomWBW CP/M 2.2 BIOS. The use of SUBMIT files is documented in Section 1.6.7 SUBMIT Command of the CPM Manual included in the Doc/CPM folder of the RomWBW distribution.
- **Z-System (ZSDOS 1.1)** - Will run run PROFILE.SUB as a SUBMIT file if it exists in A: at startup. Works exactly the same as CP/M 2.2. The original Z-System ZSDOS 1.1 did not have this ability – it was added to the RomWBW Z-System BIOS. The Z-System documentation does not cover the use of SUBMIT files – please refer to the CP/M 2.2 documentation.
- **NZCOM** - Will run the command STARTZCM at startup. This is normally an alias file, which you can edit using SALIAS. Please see Section 3.1 Creating an Alias of the NZCOM Users Manual included in the Doc/CPM folder of the RomWBW distribution.

Do not modify this file unless you fully understand the NZCOM boot process. Note that NZCOM itself is launched from ZSDOS via the included PROFILE.SUB file.

- **CP/M 3** - Will run PROFILE.SUB as a SUBMIT file if it exists in A: at startup. This mechanism is built into the CP/M 3 operating system.
  Please see Section 4.5 Executing Multiple Commands and Section 5.2.74 Executing the SUBMIT Command of the CPM3 Users Guide included in the Doc/CPM folder of the RomWBW distribution.
- **Z3PLUS** - Will run the command STARTZ3P at startup. This is normally an alias file, which you can edit using SALIAS. Please see Section 3.1 Creating an Alias of the Z3PLUS Users Manual included in the Doc/CPM folder of the RomWBW distribution. Do not modify this file unless you fully understand the Z3PLUS boot process. Note that Z3PLUS itself is launched from CP/M 3 via the included PROFILE.SUB file.
- **ZPM3** - Will run the command STARTZPM at startup. This is normally an alias file. You use SALIAS to edit such files. ZPM3 has no real documentation. The NZCOM documentation of STARTZCM is generally correct for ZPM3.

Since RomWBW can utilize many disk slices, it is very easy to create slices for specific workflows (editing, software development, games, etc.). You can then just boot to the slice that is optimized for the task you want to perform. Each such slice may have its own startup command batch file that customizes the environment for the specific workflow desired.

2.3 System Management

2.3.1 Listing Device Inventory

The device units available in your system are listed in the boot messages. However, if that list has scrolled off of your screen, you can use the 'D' command to display a list of them at any time from the Boot Loader prompt.

```
Unit        Device      Type              Capacity/Mode
----------  ----------  ----------------  --------------------
Char 0      ASCI0:      RS-232            38400,8,N,1
Char 1      ASCI1:      RS-232            38400,8,N,1
Char 2      UART0:      RS-232            38400,8,N,1
Char 3      UART1:      RS-232            38400,8,N,1
Char 4      UART2:      RS-232            38400,8,N,1
Char 5      UART3:      RS-232            38400,8,N,1
Char 6      TERM0:      Terminal          Video 0,ANSI
Char 7      PRPCON0:    Terminal          Term Module,ANSI
```

```
Disk 0      MD0:        RAM Disk          352KB,LBA
Disk 1      MD1:        Flash Drive       384KB,LBA
Disk 2      FD0:        Floppy Disk       3.5",DS/HD,CHS
Disk 3      FD1:        Floppy Disk       3.5",DS/HD,CHS
Disk 4      IDE0:       CompactFlash      3815MB,LBA
Disk 5      IDE1:       Hard Disk         --
Disk 6      IDE2:       CompactFlash      3823MB,LBA
Disk 7      IDE3:       Hard Disk         --
Disk 8      IDE4:       Hard Disk         --
Disk 9      IDE5:       Hard Disk         --
Disk 10     SD0:        SD Card           --
Disk 11     PRPSD0:     SD Card           15193MB,LBA
Video 0     TMS0:       CRT               Text,40x24
Sound 0     SND0:       AY-3-8910         3+1 CHANNELS
```

2.3.2 Rebooting the System

The 'R' command within the Boot Loader performs a software reset of the system. The system will perform a startup just like powering up or pressing the hardware reset button (although the hardware is not physically reset).

There is generally no need to do this, but it can be convenient when you want to see the boot messages again or ensure your system is in a clean state.

```
Boot [H=Help]: r

Restarting System...
```

2.3.3 Setting NVRAM Options

On systems with RTC devices (that have Non-Volatile RAM), RomWBW supports storing some limited configuration option options inside this NVRAM.

Several configuration options are currently supported, these are known as Switches

- Specify Automatic boot at startup, after an optional delay (AB)
- Define the Disk or ROM App to be booted at for automatic boot (BO)

RomWBW uses bytes located at the start of RTC NVRAM, and includes a checksum of the bytes in NVRAM to check for integrity before using the configuration.

Initially NVRAM has to be reset (with default values), before it can be used. As well as setting defaults, it also writes the correct checksum, and allows the NVRAM to be accessed and to

store the RomWBW config.

This is an explicit step that must be done, as any existing data stored is overwritten. If you are using NVRAM for other purposes, then you can continue to do so so long as you do NOT perform this Reset step.

NVRAM may also need to be reset in these circumstances:

- When there has been a loss of power to the NVRAM.
- When upgrading to a new RomWBW version, or a RomWBW version that has new switches.
- If the NVRAM has been overwritten by another application.

If you want to continue to use NVRAM in your applications you may want to consider storing your data above the RomWBW Switch data.

To configure these options an inbuilt ROM application is provided which can be accessed by the command "`W`" from the RomWBW boot menu.

This application is also built as a CP/M utility, but is not included on an disk image, it is found in the `Binary/Applications` folder of the RomWBW distribution.

For further guidance on using this application please see the section "RomWBW System Configuration" in the RomWBW Applications document.

If your system has both a Front Panel as well as NVRAM, be aware that the Front Panel switches take precedence over the NVRAM configuration settings.

Note that the WizNet class of Network devices also contain NVRAM which is entirely separate from the RomWBW configuration NVRAM described here. A separate utility is used to set the WizNet NVRAM (see CP/NET Client Setup).

RomWBW Applications

2.3.4 Changing Console and Console Speed

Your system can support a number of devices for the console. They may be VDU type devices or serial devices. If you want to change which device is the console, the ***I*** menu option can be used to choose the unit and its speed.

The command format is `I <unit> [<baudrate>]`

where ***<unit>*** is the character unit to select and ***<baudrate>*** is the optional baud rate.

Supported baud rates are:

```
 75   450  1800   7200  38400  115200   460800  1843200
150   600  2400   9600  28800  153600   614400  2457600
225   900  3600  14400  57600  230400   921600  3686400
300  1200  4800  19200  76800  307200  1228800  7372800
```

Here is an example of changing the console to unit #1 (the second serial port) and switching the port to 9600 baud:

```
Boot [H=Help]: i 1 9600

  Change speed now. Press a key to resume.

  Console on Unit #1
```

At this point, the Boot Loader prompt will be displayed on character unit #1.

Note that not all character devices support changing baud rates and some only support a limited subset of the baud rates listed. If you attempt to select an invalid baud rate for your system, you will get an error message.

2.3.5 HBIOS Diagnostic Verbosity

The 'V' command of the Boot Loader allows you to view and optionally change the level of diagnostic messages that RomWBW will produce. The normal verbosity level is 4, which means to display only fatal errors. You can increase this level to see more warnings when function calls to RomWBW HBIOS detect problems.

The use of diagnostic levels above 4 are really intended only for software developers. I do not recommend changing this under normal circumstances.

2.4 Console Takeover

If your system has more than one character unit, then the Boot Loader will "poll" all of the character devices for a request to make any of the alternate character devices the active console. This is called a console takeover request. This functionality must be enabled in the ROM build configuration, but currently it is for all standard ROMs.

To request a console takeover, you just press the <space> character twice in a row at the port or terminal that you want to move the console to. The terminal or communication software **must** be configured for the default serial port speed and data bits for this to work.

A takeover request is only possible while the active console is showing the Boot Loader prompt prior to typing any characters at the active console. In other words, once you start typing

at the active console prompt, the takeover polling is suspended. If you have started typing characters, you can press <enter> at the active console to get a fresh Boot Loader prompt and reactivate the polling.

If you have built a custom ROM that includes an automatic boot command with a timeout, then performing a console takeover will abort the timeout process and the automatic boot command will not be performed.

2.5 Front Panel

RomWBW supports the concept of a simple front panel. The following image is a conceptual view of such a front panel. If your system has a front panel, it should look similar to the RomWBW Front Panel.

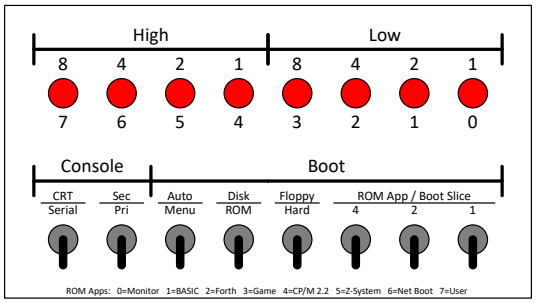


Figure 2.1: RomWBW Front Panel

The LEDs in the top row of the panel are used for multiple purposes. They are initially used to display the progress of the system boot. This may help in diagnosing a hardware or configuration issue in a system that does not progress far enough to display text output on the console. The meaning of the LEDs is:

0-------	System Boot has started
00------	Common RAM bank activated
000-----	HBIOS transitioned to RAM
0000----	Basic initialization done
00000---	CPU detection complete
000000--	System timer configured
0000000-	Pre-console device init done
00000000	Console activation

Once the system has booted, the LEDs are used to indicate disk device activity. Each LED

numbered 7-0 represents disk units 7-0. As each disk device performs I/O, the LED will light while the disk is active. This is only possible for the first 8 disk units.

The second row of the front panel is composed of switches that allow you to control a few aspects of the system startup. The switches are also re-read during warm boot, so confirm the switch settings are your desired boot configuration before executing 'REBOOT /W'.

The first two switches affect the device used as the initial console.

CRT/Serial	Sec/Pri	Explanation
CRT	Pri	Primary Cathode Ray Tube device (high speed console)
CRT	Sec	Secondary Cathode Ray Tube device (high speed console)
Serial	Pri	Boot Console is Primary Serial Port
Serial	Sec	Boot Console is Secondary Serial Port

If not available (default): "Char Unit 0" is used

The final six switches allow you to select the desired boot device.

Auto / Menu	Disk / ROM	Floppy / Hard	ROM/App/Boot Slice 4	2	1	Explanation
Menu	–	–	–	–	–	Boot to RomWBW Menu
Auto	Disk	Floppy	#	#	#	Boot Selected Floppy Disk Index
Auto	Disk	Hard	#	#	#	Boot Selected Hard Disk Index
Auto	ROM	–	#	#	#	Boot selected ROM Index

If not available: Boot Loader Command Prompt
'–' Ignored
'#' bit flag to select any three bits 4,2,1

ROM or Hard Disk - First 8 Drive Images

4 2 1	Floppy / Hard Drive Index	ROM Index
0 0 0	Zero	Monitor
0 0 1	One	BASIC
0 2 0	Two	Forth
0 2 1	Three	Game
4 0 0	Four	CP/M

4 2 1	Floppy / Hard Drive Index	ROM Index
4 0 1	Five	Z-System
4 2 0	Six	Net Boot
4 2 1	Seven	User

Chapter 3

Disk Management

The systems supported by RomWBW all have the ability to use persistent disk media. Some systems have disk interfaces built-in, while others will require add-in cards. You will need to refer to the documentation for your system for your specific options.

The RomWBW firmware provides a hardware abstraction layer, All disks will work on all hardware variations. This means you can remove disk media from one RomWBW system and put it in a different RomWBW system. The only constraint is that the applications on the disk media must be up to date with the firmware on the system being used.

3.1 Key Terminology/Concepts

The following important terminology applies

Disk (or Disk Drive)

The hardware storage device that RomWBW talks to. RomWBW supports a variety of storage disk device types.

- ROM Disk - RomWBW firmware - containing CPM utilities
- RAM Disk - A section of system RAM initially empty
- Floppy Disk(s) - Removable media
- Hard Disk(s) - Includes CF Cards, SD Cards, USB Stick, etc.

The different disk types are further defined in the section Disk Types.

Media

The storage device inserted into a disk drive, e.g. a floppy disk, CF Card, SD Card, etc.

Slice

For hard disks a Slice is a smaller logical block of disk space that is allocated and formatted with a filesystem and typically allocated to a Drive letter. Slices allow large modern storage media to be sliced up into smaller units compatible with CP/M. Slices should not be confused with partitions, a slice is not a partition.

The concept of slices is described in detail in the section Hard Disk Slices.

Drive

The mapping of a hardware disk (and slice) to a Drive letter in an operating system. A Drive has a file system installed on it

Disk Image

A disk image is a predefined image of a complete CP/M filesystem, or filesystem(s), including any partition tables (for hard disk images). Each disk image has the complete set of normal applications and tools distributed with the associated operating system or application suite.

This comes in for form of a file which is suitable for copying directly onto target media, using a modern computer. It is generally easier to use these disk images than transferring files over individually. See the section Disk Images for further details.

3.2 Startup Hardware Discovery

During startup RomWBW performs detection for hardware supported by your platform. During startup you will see messages for any disk interface(s), listing device types (e.g. FD:, IDE:, SD:), and any media that has been found on these interfaces. The messages you see will vary depending on your hardware and the media you have installed.

As an example, here are the messages you might see if you have an IDE interface in your system with a single disk drive connected to the primary side of the interface:

```
IDE: IO=0x80 MODE=MK4
IDE0: 8-BIT LBA BLOCKS=0x00773800 SIZE=3815MB
IDE1: NO MEDIA
```

See RomWBW Hardware for a complete list of the different device types supported.

If you do not see drive letters assigned as expected, refer to the prior system boot messages to ensure media has been detected in the interface.

Each drive letter refers back to a specific disk hardware interface like IDE0. This is important as it is telling you what each drive letter refers to.

Mass storage disks (like IDE) will normally have multiple drive letters assigned. The extra drive letters refer to additional "slices" on the disk.

Once your system has working disk devices, they will be accessible from any operating system you choose to run. Disk storage is available whether you boot your OS from ROM or from the disk media itself.

3.3 Drive Letter Assignment

In CP/M operating systems only 16 drive letters (A:-P:) available to be assigned to disks Drive letters were generally mapped to disk drives in a completely fixed way. For example, drive A: would **always** refer to the first floppy disk drive.

RomWBW implements a much more flexible drive letter assignment mechanism so that any drive letter can dynamically be assigned to any disk device, or slice of media.

For clarification, CP/M **cannot** refer directly to disk devices, CP/M only understands drive letters, so to access a given disk device it must first be assigned to a drive letter.

As the operating system starts up, you should see a list of drive letters assigned to the disk media you have installed. Here is an example of the list of drive letter assignments made during the startup of Z-System:

```
Configuring Drives...

        A:=MD0:0
        B:=MD1:0
        C:=FD0:0
        D:=FD1:0
        E:=IDE0:0
        F:=IDE0:1
        G:=IDE0:2
        H:=IDE0:3
```

Above you can see that:

- Drive A: has been assigned to MD0 which is the RAM disk device.
- Drive B: has been assigned to MD1 which is the ROM disk device.
- Drives C: and D: have been assigned to floppy disk drives.
- Drives E: thru H: have been assigned to the IDE0 hard disk device. The 4 entries for IDE0 are referring to the first 4 slices on that disk.

CP/M 3 and ZPM3 do not automatically display the assignments at startup, but you can use `ASSIGN` to display them.

The assignments at boot will stay the same on each boot as long as you do not make changes to your hardware configuration. i.e. If you insert or remove an SD Card, CF Card or USB Stick, the drive assignments will change at next boot.

Since drive letter assignments can change, you must be careful when doing destructive things like using `CLRDIR` to make sure the drive letter you use is referring to the desired media.

Drive assignments can be changed at any time, by the `ASSIGN` command. Please see the section Assign Drive Letters for further details.

3.3.1 Default Drive Letters

When an operating system is booted, RomWBW will automatically assign drive letters to disk devices. The assignment process varies depending on:

- the disk/slice you choose to boot from, and
- the number, type, and sizes of mass storage devices available at boot

The A: drive letter is considered special by most CP/M operating systems and is automatically used in some cases. e.g. submitting batch files, and is expected to be a writable volume.

If you boot to a physical disk device, then the first drive letter (A:) will be assigned to the disk/slice that you are booting from, and the rest of the drive letters will be offset to accommodate this. By making the selected disk/slice the A: drive, you can setup different disks/slices for specific uses and just boot it, and the booted operating system will be the A: drive

However when performing a ROM boot of an operating system, the first two drive letters will be assigned to your RAM disk (A:) and your ROM disk (B:). This provides the maximum compatibility with CP/M.

After the first drive letter is assigned (as well as the second drive letter in the case of a ROM boot), RomWBW will assign additional drive letters based on the disk drives in the system. Additional drive letters will be assigned in the following order:

- RAM Disk
- ROM Disk

- Floppy Disk(s)
- Hard Disk(s)

If a disk/slice was already assigned as the A: (or B:) drive letter, then it will not be assigned again.

Floppy or removable disk drives will be assigned a drive letter regardless of whether there is any media inserted at the time of boot.

In the case of hard disks, 1-8 drive letters will be assigned to the initial 1-8 slices of the disk drive. The number of drive letters assigned to each hard disk depends on the number of hard disks in the system:

- 1 Hard Disk: 8 drive letters (slices)
- 2 Hard Disks: 4 drive letters (slices) per disk
- 3+ Hard Disks: 2 drive letters (slices) per disk

This somewhat complicated algorithm is used to try and maximize the limited number of operating system drive letters available (16) to the available disk devices as evenly as possible.

For hard disk devices which are treated as non-removable media, drive letters will only be assigned to disk devices that actually contain media. i.e. No drive letters will be assigned to an SD Card slot that has no SD Card inserted.

3.3.2 Assign Drive Letters

The `ASSIGN` command is used to view or change the current drive letter mappings. Any changes made to drive letter mappings take immediate effect

Refer to RomWBW Applications for more information on use of the `ASSIGN` command.

Since drive letter assignments are easily changed at any time using the `ASSIGN` command, you can customize your assignments as desired after starting the operating system. Even better, you can use an auto-submit batch file to customzie the assignments at startup without any user intervention.

Chapter 4

Disk Types

4.1 RAM & ROM Disks

A typical RomWBW system has 512KB of ROM and 512KB of RAM. Some portions of each are dedicated to loading and running applications and operating system. The space left over is available for an operating system to use as a pseudo-disk device (ROM Disk and RAM Disk).

Unlike other types of disk devices, ROM and RAM Disks do not contain an actual operating system and are not “bootable”. However, they are accessible to any operating system (whether the operating system is loaded from ROM or a different disk device).

Neither RAM nor ROM disks require explicit formatting or initialization. ROM disks are pre-formatted and RAM disks are formatted automatically with an empty directory when first used.

RAM Disk

The RAM disk provides a small CP/M filesystem that you can use for the temporary storage of files. Unless your system has a battery backed mechanism for persisting your RAM contents, the RAM disk contents will be lost at each power-off.

The RAM disk is an excellent choice for storing temporary files because it is very fast. You will notice that the first time an operating system is started after the power was turned off, you will see a message indicating that the RAM disk is being formatted. If you reset your system without turning off power, the RAM disk will not be reformatted and it’s contents will still be intact.

ROM Disk

Like the RAM disk, the ROM disk also provides a small CP/M filesystem, but it's contents are static – they are part of the ROM. As such, you cannot save files to the ROM disk. Any attempt to do this will result in a disk I/O error.

The contents of the ROM disk have been chosen to provide a core set of tools and applications that are helpful for either CP/M 2.2 or ZSDOS. Since ZSDOS is CP/M 2.2 compatible, this works fairly well. However, you will find some files on the ROM disk that will work with ZSDOS, but will not work on CP/M 2.2. For example, `LDDS`, which loads the ZSDOS date/time stamper will only run under ZSDOS.

Flash ROM Disks

The limitation of ROM disks being read-only can be overcome on some platforms with the appropriate selection of Flash ROM chip and system configuration. In this case the flash-file system can be enabled which will allow the ROM disk to be read and written to. Flash devices have a limited write lifespan and continual usage will eventually wear out the device. It is not suited for high usage applications. Enabling ROM disk writing requires building a custom ROM.

4.2 Floppy Disks

If your system has the appropriate hardware, RomWBW will support the use of floppy disks. The supported floppy disk formats are generally derived from the IBM PC floppy disk formats:

- 5.25" 360K Double-sided, Double-density
- 5.25" 1.2M Double-sided, High-density
- 3.5" 720K Double-sided, Double-density
- 3.5" 1.44M Double-sided, High-density

When supported, RomWBW is normally configured for 2 3.5" floppy disk drives. If a high-density disk drive is used, then RomWBW automatically detects and adapts to double-density or high-density media. It cannot automatically detect 3.5" vs. 5.25" disk drive types – the ROM must be pre-configured for the disk drive type.

WARNING: Some of the operating systems provided with RomWBW require that a soft-reset be performed when swapping floppy disk media. For example, under CP/M 2.2, you must press control-C at the CP/M prompt after inserting a new floppy disk. The consequences of failing to perform the soft-reset vary from unexpected error messages to full disk directory corruption.

4.3 Hard Disks

The concept of a hard disk in RomWBW applies to any storage device that provides at least 8MB of space. The actual media can be a real spinning hard disk, a CompactFlash Card, a SD Card, etc. In this document, the term hard disk will apply equally to all of these.

The vintage operating systems included with RomWBW were produced at a time when mass storage devices were quite small. CP/M 2.2 could only handle filesystems up to 8MB.

Since storage devices today are quite large, RomWBW implements a mechanism called slicing (see Hard Disk Slices) to allow up to 256 8MB CP/M filesystems on a single large storage device, where each slice can be assigned to a drive letter in CPM, and be treated as its own hard disk drive (from a CPM perspective).

In order to achieve compatibility across all of the operating systems supported by RomWBW, the hard disk filesystem format used is 8MB. This ensures any filesystem will be accessible to any of the operating systems.

RomWBW uses Logical Block Addressing (LBA) to interact with all hard disks. The RomWBW operating systems use older Cylinder/Head/Sector (CHS) addressing. To accommodate the operating systems, RomWBW emulates CHS addressing. Specifically, it makes all hard disks look like they have 16 sectors and 16 heads. The number of tracks varies with the size of the physical hard disk.

4.3.1 Hard Disk Layouts

When RomWBW uses a hard disk, it utilizes an area of the physical hard disk drive space to store a sequential series of slices that contain the actual CP/M filesystems referred to by drive letters by the operating system.

Two hard disk layout schemes exist:

- Modern (hd1k)
- Classic (hd512)

You **cannot** mix disk layouts on a single disk device, however It is perfectly fine for one system to have multiple hard disks with different layouts – each physical disk device is handled separately.

If you are setting up a new disk, the Modern (hd1k) layout is recommended for the following reasons:

- Larger number of directory entries per filesystem
- Simplifies creation of coresident FAT filesystem

- Reduces chances of data corruption
- Each slice occupies exactly 8MB (an exact power of 2) in size

Both the classic and modern disk layouts continue to be fully supported by RomWBW. There are no plans to deprecate the classic layout.

Modern Layout

RomWBW (Starting with v3.2) supports the use of disk partitioning, utilising a Master Boot Record (MBR) partition tables. The Wikipedia article on the Master Boot Record is excellent if you are not familiar with them.

RomWBW uses the partition type id `0x2E`. This partition type id does not equate to any existing well-known partition types – it was chosen because it is not generally used. RomWBW does not support extended partitions – only a single primary partition can be used.

The existence of a partition table entry for RomWBW on a hard disk makes it behaves in the modern disk layout mode. Removing the RomWBW partition entry from a modern hard disk layout will cause the existing data to be unavailable and/or corrupted

The CP/M filesystem in the slices of the modern disk layout contain 1024 directory entries.

Classic Layout

Originally, RomWBW always used the very start of the hard disk media for the location of the slices. In this layout, slice 0 referred to the first chunk of ~8MB on the disk, slice 1 referred to the second chunk of ~8MB on the disk, and so on. The number of slices is limited to the size of the disk media – if you attempted to read/write to a slice that would exceed the disk size, you would see I/O errors.

The classic disk layout takes steps to allow a partition table to still be used for other types of filesystems such as DOS/FAT. It just does not use a partition table entry to determine the start of the RomWBW slices.

The lack of a RomWBW partition table entry will cause the classic disk layout to be used. Adding a partition table entry on an existing classic RomWBW hard disk will cause the existing data to be unavailable and/or corrupted.

The CP/M filesystem in the slices of the classic disk layout contain 512 directory entries.

4.3.2 Hard Disk Slices

RomWBW implements a mechanism called slicing to allow multiple CP/M filesystem on a single large storage device. To say it another way, the media is “sliced up” into many CP/M

filesystems.

You cannot use slices on any media less than 8MB in size. Specifically, you cannot slice RAM disks, ROM disks, floppy disks, etc. All of these are considered to have a single slice (0)

It is very important to understand that RomWBW slices are not individually created or allocated on your hard disk. RomWBW uses a single partition on your hard disk to contain the slices. You should think of slices as just an index into a sequential set of 8MB areas that exist in this partition.

RomWBW allows up to up to 256 slices each of 8MB in size on a single large storage device. This allows the use of up to 2GB of usable space on one media device.

It is possible to create other partitions (typically FAT), for now, we are just talking about the slices within the single RomWBW partition.

4.3.3 Slice Assignment

When assigning Hard disks to drive letters you also need to assign the slice.

Referring to slices within a storage device is done by appending a `:<n>` where <n> is the device relative slice number from 0-255. For example, if you have an IDE device, it will show up as IDE0: in the boot messages meaning the first IDE device. To refer to the fourth slice of IDE0, you would type “IDE0:3”. Here are some examples:

`IDE0:0`	First slice of disk in IDE0
`IDE0:`	First slice of disk in IDE0
`IDE0:3`	Fourth slice of disk in IDE0

So, if you wanted to use drive letter L: to refer to the fourth slice of IDE0, you could use the command `ASSIGN L:=IDE0:3`. There are a couple of rules to be aware of when assigning drive letters. First, you may only refer to a specific device/slice with one drive letter at a time. Said another way, you cannot have multiple drive letters referring to a the same device/slice at the same time. Second, there must always be a drive assigned to A:. Any attempt to violate these rules will be blocked by the `ASSIGN` command.

As you see, the name of a slice does not reference the hard disk partition containing the slices. Since there can only be a single RomWBW partition containing slices on any disk, the partition is determined automatically.

RomWBW does not prevent you from assigning slices to drive letters even if the location of the slice does not fit on the physical disk. Any attempt to access a drive letter mapped to a slice that does not fit will result in an error such as “no disk” from the operating system.

For example, a 64MB CF Card (which is typically a bit smaller than 64MB) will only fit 7 slices. At startup, you will typically see 8 drive letters assigned to the CF Card. Attempting to access the last drive letter will result in a “no disk” error from the operating system.

4.3.4 Hard Disk Capacity

The exact number of CP/M filesystem slices that will fit on your specific physical hard disk can be determined as follows:

- For Modern (hd1k) disk layouts, it is 1024KB + (slices * 8192KB). Or equivalent to say 1MB + (slices * 8MB).
- For Classic (hd512) disk layouts, it is slices * 8,320KB.

WARNING: In this document KB means 1024 bytes and MB means 1048576 bytes (frequently expressed as KiB and MiB in modern terminology). In general, hard disk capacities use KB to mean 1000 bytes and MB to mean 1,000,000 bytes.

As an example, A “64MB” CF Card probably has less than 62.5MB of actual space (using the RomWBW definition that 1MB is 1048576 bytes). Such a drive will not support 8 slices. It will support 7 slices just fine because 1024KB + (7 * 8192MB) = 57MB (hd1k) or 7 * 8,320KB = 58.24MB (hd512)

Although you can use up to 256 slices per physical disk, equating to 2GB of storage this large number of slices is rarely used. It is recommended that hard disk media used with RomWBW be 1GB or greater in capacity. This will support the RomWBW Combo Disk Image (see Combo Hard Disk Image) that allows you to use 64 CP/M filesystem slices and a 384KB FAT filesystem.

Chapter 5

Disk Preparation

There are two approaches to preparing disks for use by RomWBW.

- **Manual**: Use RomWBW itself to format empty disks and then transfer files over to the disks.
- **Images**: Use a modern computer to write a pre-built disk image including files to a disk.

This section of the document describes the manual process of preparing empty disks that are ready for use by an operating system. This is somewhat analogous to partitioning and formatting of a hard disk or doing a low level format of a floppy disk.

Alternatively, you can use the pre-built RomWBW disk images to quickly create disk media that already has a large selection of files and bootable operating system slices. Using images to prepare a disk is documented in Disk Images. You do not need to follow the instructions in this section if you want to use disk images.

5.1 Floppy Disk Formatting

Floppy media must be physically formatted before it can be used. This is normally accomplished by using the supplied Floppy Disk Utility `FDU` application. This application interacts directly with your hardware and therefore you must specify your floppy interface hardware at application startup. Additionally, you need to specify the floppy disk drive and media format to use for formatting.

Refer to RomWBW Applications for more information on use of the `FDU` command.

Since the physical format of floppy media is the same as that used in a standard MS-DOS/Windows computer, you can also format floppy disk media in a standard computer. However there

are caveats:

- The directory format itself will **NOT** be compatible with CP/M OSes. You **WILL** need to use the `CLRDIR` command to reformat the directory area from CP/M. See section Clearing (Formatting) Drives
- FDU allows you to specify physical sector interleaving (defaults to 2) which will result in faster floppy disk I/O. Formatting on a modern computer may not optimize this.

Below is a sample session using `FDU` to format a 1.44M floppy disk in the first (primary) floppy disk drive:

```
B>FDU

Floppy Disk Utility (FDU) v5.8, 26-Jul-2021 [HBIOS]
Copyright (C) 2021, Wayne Warthen, GNU GPL v3

SELECT FLOPPY DISK CONTROLLER:
  (A) Disk IO ECB Board
  (B) Disk IO 3 ECB Board
  (C) Zeta SBC Onboard FDC
  (D) Zeta 2 SBC Onboard FDC
  (E) Dual IDE ECB Board
  (F) N8 Onboard FDC
  (G) RCBus SMC (SMB)
  (H) RCBus WDC (SMB)
  (I) SmallZ80 Expansion
  (J) Dyno-Card FDC, D1030
  (K) RCBus EPFDC
  (L) Multi-Board Computer FDC
  (X) Exit
=== OPTION ===> D-IDE

===== D-IDE ============<< FDU MAIN MENU >>=======================
(S)ETUP: UNIT=00  MEDIA=720KB DS/DD    MODE=POLL        TRACE=00
----------------------------------------------------------------
(R)EAD          (W)RITE         (F)ORMAT        (V)ERIFY
(I)NIT BUFFER   (D)UMP BUFFER   FDC (C)MDS      E(X)IT
=== OPTION ===> SETUP
ENTER UNIT [00-03] (00):
00: 3.5" 720KB - 9 SECTORS, 2 SIDES, 80 TRACKS, DOUBLE DENSITY
01: 3.5" 1.44MB - 18 SECTORS, 2 SIDES, 80 TRACKS, HIGH DENSITY
```

```
02: 5.25" 320KB - 8 SECTORS, 2 SIDES, 40 TRACKS, DOUBLE DENSITY
03: 5.25" 360KB - 9 SECTORS, 2 SIDES, 40 TRACKS, DOUBLE DENSITY
04: 5.25" 1.2MB - 15 SECTORS, 2 SIDES, 80 TRACKS, HIGH DENSITY
05: 8" 1.11MB - 15 SECTORS, 2 SIDES, 77 TRACKS, DOUBLE DENSITY
06: 5.25" 160KB - 8 SECTORS, 1 SIDE, 40 TRACKS, DOUBLE DENSITY
07: 5.25" 180KB - 9 SECTORS, 1 SIDE, 40 TRACKS, DOUBLE DENSITY
08: 5.25" 320KB - 8 SECTORS, 1 SIDE, 80 TRACKS, DOUBLE DENSITY
09: 5.25" 360KB - 9 SECTORS, 1 SIDE, 80 TRACKS, DOUBLE DENSITY
ENTER MEDIA [00-09] (00): 01
00: POLLING (RECOMMENDED)
01: INTERRUPT (!!! READ MANUAL !!!)
02: FAST INTERRUPT (!!! READ MANUAL !!!)
03: INT/WAIT (!!! READ MANUAL !!!)
04: DRQ/WAIT (!!! NOT YET IMPLEMENTED!!!)
ENTER MODE [00-04] (00):
ENTER TRACE LEVEL [00-01] (00):

===== D-IDE ===========<< FDU MAIN MENU >>======================
(S)ETUP: UNIT=00  MEDIA=1.44MB DS/HD   MODE=POLL        TRACE=00
----------------------------------------------------------------
(R)EAD          (W)RITE         (F)ORMAT        (V)ERIFY
(I)NIT BUFFER   (D)UMP BUFFER   FDC (C)MDS      E(X)IT
=== OPTION ===> FORMAT (T)RACK, (D)ISK ===> DISK
ENTER INTERLEAVE [01-12] (02):

RESET DRIVE...
PROGRESS: TRACK=4F HEAD=01 SECTOR=01

===== D-IDE ===========<< FDU MAIN MENU >>======================
(S)ETUP: UNIT=00  MEDIA=1.44MB DS/HD   MODE=POLL        TRACE=00
----------------------------------------------------------------
(R)EAD          (W)RITE         (F)ORMAT        (V)ERIFY
(I)NIT BUFFER   (D)UMP BUFFER   FDC (C)MDS      E(X)IT
=== OPTION ===> EXIT
```

You can confirm a floppy disk is ready for content by simply running a `DIR` command on it. The `DIR` command should complete without error and should list no files.

5.2 Hard Disk Preparation

This section deals with initializing hard disk media entirely from your RomWBW system. The following instructions are one way to proceed. This does not mean to imply it is the only possible way.

First you need to understand

- The disk layout approach (either the Modern hd1k or the Classic hd512). See Hard Disk Layouts section if you are not sure. hd1k should be the preferred layout.
- The number of 8MB slices that you want to allocate, preferred is 64 slices. At least 1 slice of 8MB is required
- If you want to leave space for a FAT partition. See FAT Filesystem Preparation
- The total capacity of your drive, to hold the CP/M (and other) partition(s)

Then you will need to start by inserting the disk media, booting RomWBW, and confirming that the media is being recognized. If RomWBW recognizes the media, it will indicate this in the boot messages even though the media may not yet been prepared for use.

Then launch either CP/M 2.2 or Z-System from ROM using the Boot Loader `C` or `Z` commands respectively. You can now use the tools on the ROM disk to prepare your disks.

Note that you may see the operating system assign disks/slices to drives letters even though the disks/slices are not yet initialized. This is normal and does not mean the disks/slices are ready to use.

5.2.1 Partition Setup

To setup a partition you must run the `FDISK80` utility. After FDISK80 starts, enter the disk unit number of the new media. The disk unit number was assigned at boot See Device Unit Assignments

Refer to RomWBW Applications for more information on use of the `FDISK80` utility.

If you want to use the Classic (hd512) layout skip down to the Classic (hd512) section

Modern (hd1k)

At this point, use the `I` command to initialize (reset) the partition table to an empty state.

You must create a partition for the RomWBW CP/M slices. Then create a partition using the `N` command. Importantly:

- The partition number should typically be `1` the first partition

- The partition can be placed anywhere you want, The typical location for the RomWBW partition is at 1MB.
- The partition size should be the total size of all the slices you require, and must be at least 8MB in size, in increments of 8MB makes sense.

You **must** then set the type of partition to `2E` using the `T` command. The `P` command can be used to display the partition before it is written Finally the partition can be written to disk using the `W` write command.

WARNING: Modifying the partition table of existing media will make any data on the media inaccessible.

Below is an example of creating a RomWBW partition following these guidelines.

```
FDISK80 for RomWBW, UNA, Mini-M68k, KISS-68030, SBC-188  ----
       Version 1.1-22 created 7-May-2020
                 (Running under RomWBW HBIOS)

HBIOS unit number [0..11]: 4
Capacity of disk 4:  (  4G)  7813120      Geom 77381010
Nr  ---Type- A --      Start         End   LBA start  LBA count  Size
 1             00       *** empty ***
 2             00       *** empty ***
 3             00       *** empty ***
 4             00       *** empty ***
>>i
>>n
New partition number: 1
Starting Cylinder (default 0): 1Mb
Ending Cylinder (or Size= "+nnn"): +512Mb
>>t
Change type of partition number: 1
New type (in hex), "L" lists types: 2e
>>p
Nr  ---Type- A --      Start         End   LBA start  LBA count  Size
 1    RomWBW   2e      8:0:1  1023:15:16        2048    1048576  512M
 2             00       *** empty ***
 3             00       *** empty ***
 4             00       *** empty ***
>>w
Do you really want to write to disk? [N/y]: y
Okay
```

```
FDISK exit.
```

At this point, it is best to restart your system to make sure that the operating system is aware of the partition table updates. Start CP/M 2.2 or Z-System from ROM again.

Classic (hd512)

At this point, use the `I` command to initialize (reset) the partition table to an empty state.

To use the hd512 layout, use `W` to write the empty table to the disk and exit. Remember that the lack of a partition for RomWBW implies the Classic (hd512) layout.

At this point, it is best to restart your system to make sure that the operating system is aware of the partition table updates. Start CP/M 2.2 or Z-System from ROM again.

FAT Partition

At this point you may want to consider creating a FAT partition Please see the section FAT Filesystem Preparation for details on how to setup the FAT partition.

5.2.2 Slice Initialization

You need to initialize each slice for CP/M to use it. This is somewhat analogous to doing a FORMAT operation on other systems, and is done using the `CLRDIR` command.

This is covered in the next section Clearing (Formatting) Drives

Assuming you want to use additional slices, you should initialize them using the same process. You may need to reassign drive letters to access some slices that are beyond the ones automatically assigned. You can use the `ASSIGN` command to handle this.

5.3 Post Disk Preparation

Once a disk has been initialised you may need to do one or more of the following; * Clear (Format) the drive * Make the disk bootable * Copy system (or other) files to the disk

5.3.1 Clearing (Formatting) Drives

This is somewhat analogous to doing a FORMAT operation on other systems.

With RomWBW you use the `CLRDIR` command to do this. This command is merely “clearing out” the directory space of the drive referred to by a drive letter and setting up the new empty directory.

Refer to RomWBW Applications for more information on use of the CLRDIR command.

Since CLRDIR works on drive letters, make absolutely sure you know what media and slice are assigned to that drive letter before using CLRDIR because CLRDIR will wipe out any pre-existing contents of the slice.

After CLRDIR completes, the drive should be ready to use by the operating system via the drive letter assigned. Start by using the DIR command on the drive. This should return without error, but list no files.

Here is an example of using CLRDIR. In this example, the ASSIGN command is used to show the current drive letter assignments. Then the CLRDIR command is used to initialize the directory of drive 'G' which is slice 2 of hard disk device IDE0 ("IDE0:2").

```
B>ASSIGN

   A:=MD0:0
   B:=MD1:0
   C:=FD0:0
   D:=FD1:0
   E:=IDE0:0
   F:=IDE0:1
   G:=IDE0:2
   H:=IDE0:3

B>CLDIR G:
CLRDIR Version 1.2B May 2024 by Max Scane

Warning - this utility will overwrite the directory sectors of Drive: G
Type CAPITAL Y to proceed, any key other key to exit. Y
Directory cleared.
B>
```

5.3.2 Making a Disk Bootable

To make a disk bootable you will need to follow the specific instructions in Operating Systems, as each operating system will be different.

Generally you will need to use SYSCOPY to setup the system track(s) of the disk.As an example, If you wanted to setup C: as a bootable Z-System disk you would use:

```
B>SYSCOPY C:=B:ZSYS.SYS
```

```
SYSCOPY v2.0 for RomWBW CP/M, 17-Feb-2020 (CP/M 2 Mode)
Copyright 2020, Wayne Warthen, GNU GPL v3

Transfer system image from B:ZSYS.SYS to C: (Y/N)? Y
Reading image... Writing image... Done
```

Once this process succeeds, you will be able to boot directly to the disk slice from the boot loader prompt. See the instructions in Starting Operating Systems from Disk for details on this.

5.3.3 Copying System Files

As well as making the disk bootable, you may need to transfer other system and application files to your disks. Refer to Transferring Files for more information on getting files onto your disks.

5.3.4 Checking Disk Layout

If you are not sure which disk layout is used for your existing media, you can use the CP/M 2.2 `STAT` command to display information including the number of "32 Byte Directory Entries" for a drive letter on the corresponding hard disk.

Note: For CP/M 3 the command is `SHOW [DRIVE]`

- If it indicates 512, your disk layout is Classic (hd512).
- If it indicates 1024, your disk layout is Modern (hd1k).

Here is an example of checking the disk layout.

```
B>STAT E:DSK:

    E: Drive Characteristics
65408: 128 Byte Record Capacity
 8176: Kilobyte Drive  Capacity
 1024: 32  Byte Directory Entries
    0: Checked  Directory Entries
  256: Records/ Extent
   32: Records/ Block
   64: Sectors/ Track
    2: Reserved Tracks
```

It is critical that you include `DSK:` after the drive letter in the `STAT` command line. The important line to look at is labeled "32 Byte Directory Entries".

Chapter 6

Disk Images

Since it would be quite a bit of work to transfer over all the files you might want initially to your disk(s), It is generally easier to use these disk images than transferring your files over individually. RomWBW comes with a variety of ready to use disk images.

You can use your modern Windows, Linux, or Mac computer to copy a disk image onto the disk media, then just move the media over to your RomWBW computer.

The disk image files are found in the Binary directory of the distribution.

Each disk image has the complete set of normal applications and tools distributed with the associated operating system or application suite. The following table shows the disk images available.

Disk Image	Description	Boot
xxx_aztec.img	Aztec C Compiler	No
xxx_bascomp.img	Microsoft Basic-80 Compiler	No
xxx_blank.img	*empty image*	No
xxx_cowgol.img	Cowgol 2.0 Compiler	No
xxx_cpm22.img	DRI CP/M 2.2 Operating System	Yes
xxx_cpm3.img	DRI CP/M 3 Operating System	Yes
xxx_dos65.img	DOS/65 Operating System	Yes
xxx_fortran.img	Microsoft Fortran-80 Compiler	No
xxx_games.img	Games Disk for CP/M	No
xxx_hitechc.img	HI-TECH Z80 CP/M C compiler	No
xxx_infocom.img	Infocom Games Disk	No
xxx_nzcom.img	NZCOM ZCPR 3.4 Operating System	Yes
xxx_qpm.img	QPM Operating System	Yes

Disk Image	Description	Boot
xxx_tpascal.img	Borland Turbo Pascal Compiler	No
xxx_wp.img	WordStar v4 & ZDE Applications	No
xxx_z3plus.img	Z3PLUS ZCPR 3.4 Operating System	Yes
xxx_z80asm.img	Relocating macro assembler for CP/M	No
xxx_zpm3.img	ZPM3 Operating System	Yes
xxx_zsdos.img	ZCPR-DJ & ZSDOS 1.1 Operating System	Yes

You will find 3 sets of these .img files in the distribution. The “xxx” portion of the filename will be:

- “fd_” for a floppy image.
- “hd1k_” for a Modern layout hard disk image.
- “hd512_” for a Classic layout hard disk image.

In the case of xxx_dos65.img, only an hd512 variant is provided. This is a constraint of the DOS65 distribution.

There is also an image file called “psys.img” which contains a bootable p-System hard disk image. It contains 6 p-System filesystem slices, but these are not interoperable with the CP/M slices described above. This file is discussed separately under UCSD p-System in Operating Systems section.

6.1 Floppy Disk Images

The floppy disk images are all intended to be used with 3.5” high-density, double-sided 1.44 MB floppy disk media. This is ideal for the default floppy disk drive support included in RomWBW standard ROMs.

For floppy disks, the .img file is written directly to the floppy media as is. The floppy .img files are 1.44 MB which is the exact size of a single 3.5” high density floppy disk. You will need a floppy disk drive of the same type connected to your modern computer to write this image. Although modern computers do not come equipped with a floppy disk drive, you can still find USB floppy drives that work well for this.

The floppy disk must be physically formatted **before** writing the image onto it. You can do this with RomWBW using FDU as described in the Floppy Disk Formatting section of this document. While you can also format the floppy using your modern computer, using FDU is preferable.

RomWBW includes a Windows application called RawWriteWin in the Tools directory of the distribution. This simple application will let you choose a file and write it to an attached floppy

disk drive. For Linux/MacOS, I think you can use the dd command (but I have not actually tried this). It is probably obvious, but writing an image to a floppy disk will overwrite and destroy all previous contents.

Once the image has been written to the floppy disk, you can insert the floppy disk in your RomWBW floppy disk and read/write files on it according to the specific operating system instructions. If the image is bootable, then you will be able to boot from it by entering the floppy disk drive's corresponding unit number at the RomWBW Boot Loader command prompt.

6.2 Hard Disk Images

Keeping in mind that a RomWBW hard disk (including CF/SD/USB devices) allows you to have multiple slices (CP/M filesystems), there are a couple ways to image hard disk media. The easiest approach is to use the Combo Disk Image. This hard disk image is already prepared with 6 slices containing 5 ready-to-run OSes and a slice with the WordStar application files.

Alternatively, you can create your own hard disk image with the specific slice contents you choose.

6.2.1 Standard Hard Disk Layout

As previously described in Hard Disk Layouts, the exact placement of slices and optional FAT partition will vary depending on which disk layout (hd512 or hd1k) you are using and your partition table entries. To simplify the use of hard disk images, RomWBW has adopted a standard partition layout for disk image files provided. This standard layout is used to produce the Combo Disk Images described below.

These partition sizes and locations were chosen to:

- Fit entirely on 1GB media
- Allow for 64 CP/M filesystem slices
- Allow for a 384KB FAT filesystem

The standard partition table table entries are:

	— Modern (hd1k) —		— Classic (hd512) —	
	Byte(s)	Sector(s)	Byte(s)	Sector(s)
RomWBW Partition Start	1 MB	2,048	–	–
RomWBW Partition Size	512 MB	1,048,576	–	–
FAT Filesystem Start	513 MB	1,050,624	520 MB	1,064,960
FAT Filesystem Size	384 MB	786,432	384 MB	786,432

	— Modern (hd1k) —		— Classic (hd512) —	
	Byte(s)	Sector(s)	Byte(s)	Sector(s)
<end>	897 MB	1,851,392	904 MB	1,837,056

NOTE: RomWBW is not limited to these partition table entries. You can change the size and location of the RomWBW and/or FAT partitions to increase/decrease the number of slices or FAT filesystem size. Doing so would require using `FDISK80` to define your own custom disk layout and initializing your filesystems manually.

The RomWBW System Guide has more information on the standard disk layouts as implemented in the Combo Disk Images. Additionally, there is a document called "Hard Disk Anatomy.pdf" in the Doc directory of the RomWBW distribution with detailed information on the standard disk layouts.

6.2.2 Combo Hard Disk Image

The Combo Disk Image is essentially just a single disk image that has several of the individual filesystem images (slices) already concatenated together using the standard disk layout described above. The Combo Disk Image includes the partition table of the standard disk layout and the following 6 slices in the positions indicated:

Slice	Description
Slice 0	DRI CP/M 2.2 Operating System
Slice 1	ZCPR-DJ & ZSDOS 1.1 Operating System
Slice 2	NZCOM ZCPR 3.4 Operating System
Slice 3	DRI CP/M 3 Operating System
Slice 4	ZPM3 Operating System
Slice 5	WordStar v4 & ZDE Applications
Slice 6-63	*blank unformatted*

There are actually 2 Combo Disk Images in the distribution. One for an hd512 disk layout (hd512_combo.img) and one for an hd1k disk layout (hd1k_combo.img). Simply use the image file that corresponds to your desired hard disk layout. Review the information in Hard Disk Layouts if you need more information of the disk layout options.

> **Note**: Apart from the hd512 and hd1k Combo Disk Images (mentioned above) there are actually a number of other `hd1k_*_combo.img` files. These additional combo files are platform (generally romless) specific, and should be ignored

unless you are on one of these platforms. If you are on one of these platforms you must use the correct combo file

The Combo Disk Image actually only contains the initial partition table, and the first 6 slices (Slice 0 to 5), this is approximately 49MB in size. While the partition table reserves space to store 64 CP/M filesystem slices as well as a single 384MB FAT filesystem, these areas remain empty, and must be initialized manually using `CLRDIR` for CP/M filesystems and `FAT FORMAT` for the FAT filesystem.

Combo Disk Image Capacity

The standard hard disk layout used by the Combo Disk Image was designed to fit well on a 1GB hard disk. The 64 CP/M slices (approximately 512MB) and 384MB FAT filesystem all fit well within a 1GB hard disk. This size choice was a bit arbitrary, but based on the idea that 1GB CF/SD/USB Media is easy and cheap to acquire.

It is fine if your hard disk is smaller than 1GB. It just means that it will not be possible to use the pre-allocated FAT filesystem partition and any CP/M filesystem slices that don't fit. The true number of CP/M filesystem slices that will fit on your specific physical hard disk can be calculated as described in Hard Disk Capacity.

If you attempt to access a slice past the end of the physical hard disk you will get "no disk" errors. You should calculate the maximum number of slices your hard disk will support and do not exceed this number.

Combo Disk Image Advice

A great way to maintain your own data on a hard disk is to put your data in slices beyond the first 6. By doing so, you can always "re-image" your drive media with the Combo Disk Image without overlaying the data stored in the slices beyond the first 6. Just be very careful to use the same combo image layout (hd512 or hd1k) as you used originally.

6.2.3 Custom Hard Disk Image

For hard disks, each .img file represents a single slice (CP/M filesystem). Since a hard disk can contain many slices, you can just concatenate the slices (.img files) together to create your desired hard disk image.

If you look in the Binary directory of the distribution, you will see that there are more disk (slice) images than the 6 that are included in the Combo Disk Images. These supplemental disk images are identified by looking for the files that start with hd1k_ or hd512_.

There are two approaches you can use to create custom hard disk images with multiple slices.

- You can add/modify a configuration file and run the RomWBW build process. This requires running the RomWBW build process, but will cause your custom hard disk images to be created with every build.
- You can manually combine the individual images using `COPY` (Windows) or `cat` (Linux/MacOS). This does not require running the RomWBW build process, but will require manually recreating your custom hard disk images when you upgrade to new releases of RomWBW.

The following sections provide more detail on each approach.

Building Custom Hard Disk Images

The RomWBW build process builds the disk images defined in the `Source/Images` directory. The resultant images are placed in the `Binary` directory and are ready to copy to your media.

These aggregate disk images are defined using .def files. You will see there is a combo.def file in the Images directory that defines the slices for the Combo disk image. You can create your own .def files as desired to automatically create custom aggregate disk images. When the RomWBW build process is run, it will automatically look for all .def files in the `Source/Images` directory and create aggregate disk images for each using the same base name as the .def file.

There is an example of this in the `Images` directory called `all.def.example`. You can remove the ".example" suffix so that the file is called `all.def`. Now, if you run the RomWBW build process, it will automatically generate `hd512_all.img` and `hd1k_all.img` files in the `/Binary` directory. This example creates an aggregate disk image with all of the possible slices.

You could also modify the contents of the Combo disk image by simply modifying the `combo.def` configuration file. However, it is recommended that you leave the Combo image alone and simply define your own.

NOTE: All of the `hd1k_xxx.img` aggregate disk image files created in this way (including the Combo disk image) will already be prefixed with `hd1k_prefix.dat`, so you do not need to add the prefix file. They are ready to write to your media.

Combining Hard Disk Images Manually

You can add slices to the Combo Disk Images simply by tacking slices onto the end. For example, if you want to add a slice containing the Fortran compiler to the end of the combo image, you could use one of the following command lines depending on your operating system:

Windows:

```
COPY /B hd1k_combo.img + hd1k_fortran.img my_hd.img
```

Linus/MaxOS:

```
cat hd1k_combo.img hd1k_fortran.img >my_hd.img
```

Note that you **must** be sure to use either the hd1k_ or hd512_ prefixed files together. You cannot mix them.

Creating a new Custom Image Manually

If you want to create a completely custom hard disk image that is not based on the existing combo image, you can generate a disk image entirely from scratch using whatever slices you want in whatever order you like.

For example, if you want to create a hard disk image that has slices for CP/M 2.2, CP/M 3, and WordStar in the hd512 format, you would use the command line of your modern computer to create the final image:

Windows:

```
COPY /B hd512_cpm22.img + hd512_cpm3.img + hd512_ws hd.img
```

Linux/MacOS:

```
cat hd512_cpm22.img hd512_cpm3.img hd512_ws >hd.img
```

NOTE: For the hd1k disk layout, you **must** prepend the prefix file called hd1k_prefix.dat which contains the required partition table. So, for an hd1k layout you would use the following:

Windows:

```
COPY /B hd1k_prefix.dat + hd1k_cpm22.img + hd1k_cpm3.img + hd1k_ws hd.img
```

Linux/MacOS:

```
cat hd1k_prefix.dat hd1k_cpm22.img hd1k_cpm3.img hd1k_ws >hd.img
```

> **Note**: Apart from the hd1k_prefix.dat file (mentioned above) there are actually a number of other `hd1k_*_prefix.dat` files. These additional prefix files are platform (generally romless) specific, and should be ignored unless you are on one of these platforms. If you are on one of these platforms you must use the correct prefix file

In all of the examples above, the resulting file (hd.img) would now be written to your hard disk media and would be ready to use in a RomWBW system.

If you wish to further customize or create new disk image definitions, please refer to the ReadMe.txt file in the Source/Images directory.

6.2.4 Writing Hard Disk Images

Once you have chosen a Combo Hard Disk Image file or prepared your own custom hard disk image file, it will need to be written to the media using your modern computer. When using this method, the disk will be partitioned and setup with 1 or more slices containing ready-to-run bootable operating systems.

To write a hard disk image file onto your actual media (actual hard disk or CF/SD/USB Media), you need to use an image writing utility on your modern computer. Your modern computer will need to have an appropriate interface or slot that accepts the media. To actually copy the image

- On Linux or MacOS , you can use the `dd` command.
- On Windows, in the "Tools" directory of the distribution, there is an application called Win32DiskImager.

In all cases, the image file should be written to the media starting at the very first block or sector of the media. This is the default behaviour on all software.

To be entirely clear, writing a disk image file to your hard disk media will overwrite an pre-existing partition table and the number of slices that your image file contains. It will not overwrite or corrupt slices beyond those in the image file. As a result, you can use additional slices as a place to maintain your personal data because these slices will survive re-imaging of the media. If you setup a FAT partition on your media, it will also survive the imaging process.

Media Usage and Initialization

Once you have copied the image onto the hard disk media, you can move the media over to your RomWBW system. You can then boot to the operating system slices by specifying (***<diskunit>.<slice>***) at the RomWBW Boot Loader command prompt. See the section Starting Operating Systems from Disk for further details

You are not limited to the number of slices that are contained in the image that you write to your hard disk media. You can use additional slices as long your media has room for them.

However, writing the disk image will not initialize the additional slices. You will need to use the `CLRDIR` application to initialize any un-initialized slice before its first use, and and optionally `SYSCOPY` if you want th slice to be bootable. If you use the combo image this applies to slices 6 thru 63. The procedure for this is documented in the Clearing (Formatting) Drives section.

Likewise, the pre-allocated FAT partition must still be formatted using `FAT FORMAT` in order to actually use it (see FAT Filesystem Preparation). Alternatively, the FAT partition can be formatted on a modern computer.

Re-Imaging Existing Media

In order for your additional slices and/or FAT partition to survive re-imaging, you **must** follow these rules:

- Do not modify the partition table of the media using FDISK80 or any other partition management tools.
- Ensure that your hard disk image file uses the same disk layout approach (hd512 or hd1k) as previously used on the media.

6.2.5 Writing Hard Disk Slices

This section covers techniques to copy partial images onto pre-existing media, in effect performing a selective slice copy. These techniques currently **only** apply to hd1k formatted media, which has a partition table entry, and a convenient 1MB size metric. However adapting to hd512 is possible, but left to the user.

You will need to have access to a Linux/MacOS machine, or have the Linux tools for Windows installed, including the `dd` command line tool. For Windows there are multiple options for installing `dd` including MSYS2, CygWin, and dd for Windows.

WARNING: The `dd` command is a low-level utility that writes directly to raw disk sectors with almost no safety checks. It is very easy to corrupt a disk if this tool is used incorrectly.

The `dd` command supports options to define precisely source and destination offsets and sizes to copy. From the documentation of `dd` the following options are important.

```
if=file  Read input from file instead of the standard input.
of=file  Write output to file instead of the standard output.
skip=n   Skip n blocks from the beginning of the input before
         copying.
seek=n   Seek n blocks from the beginning of the output before
         copying.
count=n  Copy only n input blocks.
bs=n     Set both input and output block size to n bytes instead
         of the default 512
```

The best approach is to copy data to the RomWBW partition. To do this, you must first determine the name that your operating system is using for the RomWBW disk and partition.

An easy way to determine this may be the linux `mount` command, which lists the currently mounted partitions. From here you can more easily determine the naming scheme used by your operating system.

Typically disk devices are named something like `/dev/disk9` or `/dev/sdg`, noting above the `g` is a alphabetic and it could be any letter. This naming is arbitrary and depend on the operating system, and the specific hardware connecting the device

Partitions are typically named by simply adding a number after the name of the hard disk device. For example, the first partition could be `/dev/disk9s1`, `/dev/sdg1`, or `/dev/sdgp1`.

In the following examples we use the above `dd` options, noting the `of=` option is the RomWBW target partition.

** NOTE ** A second approach (Examples 3 and 4) is to address the hard disk as a raw disk device and we take explicit steps to calculate the start of the RomWBW partition. While this works, it is better to use the partition relative approach.

The commands in the examples below are run from the `Binary` folder of RomWBW distribution.

Example 1 : Copy the Games image to an empty slice of our media

In this example we will copy the (hd1k) games image to Slice 6 (free) of our existing media. In this example we assume the media has already been formatted with the combo image, which already contains 6 slices (numbered from 0 to 5). We are just copying the needed slice to the existing media as a new slice (number 6) after the existing slices making it the 7th slice.

```
Binary % sudo dd if=hd1k_games.img of=/dev/sdg1 seek=6 bs=8M

Password:

1+0 records in
1+0 records out
8388608 bytes transferred in 1.917296 secs (4375228 bytes/sec)
```

Since bs=8MB the `seek=6` skips the first 6 (8MB slices) slices (in the target image) and writes the games image to the 7th slice.

Example 2 : Copy the entire Combo Image without replacing partition table

In all of the following examples we use `bs=1MB` to defines the block size used in other parameters. This is convenient since the combo image reserves 1MB for the partition table at the start of the disk.

In this example we will copy the (hd1k) combo image (which includes a partition table) over our media without replacing the partition table. In this example we assume the media has already been formatted with the combo image, and we have modified the partition table, which we do not want to overwrite.

```
Binary % sudo dd if=hd1k_combo.img of=/dev/sdg1 skip=1 bs=1M

Password:

48+0 records in
48+0 records out
50331648 bytes transferred in 11.503776 secs (4745528 bytes/sec)
```

The `skip=1` skips the first 1MB (partition table) in the input file, effectively stripping out the combo images partition table, before overwriting the slices in the target partition.

Example 3 : Copy the Combo Image without replacing partition table

This example is identical to Example 2 except it writes to the target disk device itself (`of=/dev/disk9`), not the target partition.

```
Binary % sudo dd if=hd1k_combo.img of=/dev/disk9 skip=1 seek=1 bs=1M

Password:

48+0 records in
48+0 records out
50331648 bytes transferred in 11.503776 secs (4745528 bytes/sec)
```

The `skip=1` skips the first 1MB in the input file, and likewise `seek=1` skips the first 1MB of the target media file we are writing to, thus in effect we are skipping the first 1MB, which contains the partition table itself.

Example 4 : Copy the Games image to an empty slice of our media

In this example we will copy the (hd1k) games image to Slice 6 (free) of our existing media. In this example we assume the media has already been formatted with the combo image, which already contains 6 slices (numbered from 0 to 5) We are just coping the needed slice to this existing media as a new slice (number 6) after the existing slices making it the 7th slice.

```
Binary % sudo dd if=hd1k_games.img of=/dev/disk9 seek=49 bs=1M

Password:
```

```
8+0 records in
8+0 records out
8388608 bytes transferred in 1.917296 secs (4375228 bytes/sec)
```

The `seek=49` skips the first 49MB of the media file we are writing to. 49 is calculated as `(slice number * 8) + 1`, where 8 is the size of a slice and 1 is the size of the partition table im megabytes. Thus we are skipping 6 slices (in the combo image) and writing to the 7th slice.

Chapter 7

Operating Systems

One of the primary goals of RomWBW is to expose a set of generic hardware functions that make it easy to adapt operating systems to any hardware supported by RomWBW. As a result, there are now 8 operating systems that have been adapted to run under RomWBW. The adaptations are identical for all hardware supported by RomWBW because RomWBW hides all hardware specifics from the operating system.

By design, the operating systems provided with RomWBW are original and unmodified from their original distribution. Patches published by the authors are generally included or applied. The various enhancements RomWBW provides (such as hard disk slices) are implemented entirely within the system adaptation component of each operating system (e.g., CP/M CBIOS). As a result, each operating system should function exactly as documented by the authors and retain maximum compatibility with original applications.

Note that all of the operating systems included with RomWBW support the same basic filesystem format from DRI CP/M 2.2 (except for p-System). As a result, a formatted filesystem will be accessible to any operating system. The only possible issue is that if you turn on date/time stamping using the newer OSes, the older OSes will not understand this. The older OS will not corrupt the files, but the date/time stamps will not be maintained.

The following sections briefly describe the operating system options currently available and brief operating notes.

7.1 Digital Research CP/M 2.2

This is the most widely used variant of the Digital Research operating systems. It has the most basic feature set, but is essentially the compatibility metric for all other CP/M-like operating

systems including those listed below.

If you are new to the CP/M world, I would recommend using this CP/M variant to start with simply because it is the most stable and you are less likely to encounter compatibility issues.

Documentation

- CPM Manual

Boot Disk

To make a bootable CP/M disk, use the RomWBW `SYSCOPY` tool to place a copy of the operating system on the boot track of the disk. The RomWBW ROM disk has a copy of the boot track call "CPM.SYS". For example:

```
SYSCOPY C:=B:CPM.SYS
```

Character Device Mapping

Character device mapping under CP/M 2.2 has 3 layers:

CP/M Logical Device –> CP/M Physical Device –> RomWBW HBIOS Device

The CP/M Logical Devices are:

Device	Description
`CON:`	System console device, used by CCP for communication with the operator
`RDR:`	Paper tape reader device
`PUN:`	Paper tape punch device
`LST:`	Output list device

The CP/M Physical Devices are:

Device	Description
`TTY:`	Teletype device (slow speed console)
`CRT:`	Cathode ray tube device (high speed console)
`BAT:`	Batch processing (input from `RDR:`, output to `LST:`)
`UC1:`	User-defined console
`PTR:`	Paper tape reader (high speed reader)
`UR1:`	User-defined reader #1
`UR2:`	User-defined reader #2

Device	Description
PTP:	Paper tape punch (high speed punch)
UP1:	User-defined punch #1
UP2:	User-defined punch #2
LPT:	Line printer
UL1:	User-defined list device #1

CP/M Logical Devices are mapped to CP/M Physical Devices via the IOBYTE at 0x0003 as shown below.

Logical Device	LST:	PUN:	RDR:	CON:
IOBYTE Bits	7 6	5 4	3 2	1 0
0 (0b00)	TTY:	TTY:	TTY:	TTY:
1 (0b01)	CRT:	PTP:	PTR:	CRT:
2 (0b10)	LPT:	UP1:	UR1:	BAT:
3 (0b11)	UL1:	UP2:	UR2:	UC1:

The mappings above can be managed using the STAT command. This command essentially just modifies the IOBYTE value.

CP/M Physical Devices are mapped to RomWBW HBIOS devices during the boot process depending on the number of HBIOS Char devices in the system.

All CP/M Physical Devices are initially mapped to HBIOS Char 0. If additional HBIOS Char devices are available in the system, they will be mapped as below:

CP/M	RomWBW HBIOS
TTY:	Char 0
CRT:	CRT
BAT:	CP/M RDR/LST
UC1:	Char 1
PTR:	Char 1
UR1:	Char 2
UR2:	Char 3
PTP:	Char 1
UP1:	Char 2
UP2:	Char 3
LPT:	Char 1

CP/M	RomWBW HBIOS
UL1:	Char 2

Normally, the HBIOS Console device (Loader prompt) is on HBIOS Device Char 0. If it has been reassigned to a different HBIOS character device before launching CP/M, then the above mapping will be modified. TTY: will be assigned to the current HBIOS console Char device. The remaining assignments will be filled in with the other Char devices as available.

Notes

- You can change media, but it must be done while at the OS command prompt and you **must** warm start CP/M by pressing ctrl-C. This is a CP/M 2.2 constraint and is well documented in the DRI manual.
- `SUBMIT.COM` has been patched per DRI to always place submit files on A:. This ensures the submitted file will always be properly executed.
- The original versions of DDT, DDTZ, and ZSID used the RST 38 vector which conflicts with interrupt mode 1 use of this vector. The DDT, DDTZ, and ZSID applications in RomWBW have been modified to use RST 30 to avoid this issue.
- Z-System applications will not run under CP/M 2.2. For example, the `LDDS` date stamper will not work.

7.2 Z-System

Z-System is the most popular non-DRI CP/M workalike “clone” which is generally referred to as Z-System. Z-System is intended to be an enhanced version of CP/M and should run all CP/M 2.2 applications. It is optimized for the Z80 CPU (as opposed to 8080 for CP/M) and has some significant improvements such as date/time stamping of files.

Z-System is a somewhat ambiguous term because there are multiple generations of this software. RomWBW Z-System is a combination of both ZCPR-DJ (the CCP) and ZSDOS 1.1 (the BDOS) when referring to Z-System. The latest version of Z-System (ZCPR 3.4) is also provided with RomWBW via the NZ-COM adaptation (see below).

Documentation

- ZCPR Manual
- ZCPR-DJ

- ZSDOS Manual
- ZSDOS Programmer's Manual

Boot Disk

To make a bootable Z-System disk, use the RomWBW `SYSCOPY` tool to place a copy of the operating system on the boot track of the disk. The RomWBW ROM disk has a copy of the boot track call "ZSYS.SYS". For example:

```
SYSCOPY C:=B:ZSYS.SYS
```

Character Device Mapping

Mapping of character devices to RomWBW HBIOS Character devices operates exactly the same as described in Digital Research CP/M 2.2. The CP/M 2.2 `STAT` command is used to manipulate the device mappings.

Notes

- Although most CP/M 2.2 applications will run under Z-System, some may not work as expected. The best example is PIP which is not aware of the ZSDOS paths and will fail in some scenarios (use `COPY` instead).
- Although ZSDOS can recognize a media change in some cases, it will not always work. You should only change media at a command prompt and be sure to warm start the OS with a ctrl-C.
- ZSDOS has a concept of fast relog of drives. This means that after a warm start, it avoids the overhead of relogging all the disk drives. There are times when this causes issues. After using tools like `CLRDIR` or MAP, you may need to run "RELOG" to get the drive properly recognized by ZSDOS.
- ZSVSTAMP expects to be running under the ZCPR 3.X command processor. By default, RomWBW uses ZCPR 1.0 (intentionally, to reduce space usage) and ZSVSTAMP will just abort in this case. It will work fine if you implement NZCOM. ZSVSTAMP is included solely to facilitate usage if/when you install NZCOM.
- Many of the tools can be configured (using either ZCNFG or DSCONFIG). The configuration process modifies the actual application file itself. This will fail if you try to modify one that is on the ROM disk because it will not be able to update the image.
- DATSWEEP can be configured using DSCONFIG. However, DSCONFIG itself needs to be configured first for proper terminal emulation by using SETTERM. So, run SETTERM on DSCONFIG before using DSCONFIG to configure DATSWEEP!

- Generic CP/M PIP and ZSDOS path searching do not mix well if you use PIP to copy to or from a directory in the ZSDOS search path. Best to use COPY from the ZSDOS distribution.

7.3 NZCOM Automatic Z-System

NZCOM is a much further refined version of Z-System (ZCPR 3.4). NZCOM was sold as an enhancement for existing users of CP/M 2.2 or ZSDOS. For this reason, (by design) NZCOM does not provide a way to boot directly from disk. Rather, it is loaded after the system boots into a host OS. On the RomWBW NZCOM disk images, the boot OS is ZSDOS 1.1. A `PROFILE.SUB` file is included which automatically launches NZCOM as soon as ZSDOS loads.

NZCOM is a companion product to Z3PLUS, they are almost identical having been written by the same team. The only difference is the base operating system on which they run, but the architecture, the tools, libraries, files, etc are all primarily the same.

NZCOM is highly configurable. The RomWBW distribution has been configured in the most basic way possible. You should refer to the documentation and use `MKZCM` as desired to customize your system.

NZCOM has substantially more functionality than CP/M or basic Z-System. It is important to read the “NZCOM Users Manual.pdf” document in order to use this operating system effectively.

Documentation

- NZCOM Users Manual
- Z-System Users Guide
- ZCPR3.3 User Guide
- ZSDOS Programmer’s Manual

Boot Disk

Since NZ-COM boots via Z-System, you can make a bootable NZ-COM disk using `ZSYS.SYS` as described in Z-System above. You will need to add a `PROFILE.SUB` file to auto-start NZ-COM itself.

Character Device Mapping

Mapping of character devices to RomWBW HBIOS Character devices operates exactly the same as described in Digital Research CP/M 2.2. However, it is **not** possible to manipulate the CP/M Logical to Physical device mapping using the `STAT` command. The mapping is static.

Note: A custom ZCPR IOP module could theoretically be used to manage the character device mappings. RomWBW does not provide this module and writing an IOP module is beyond the scope of this document.

Notes

- All of the notes for Z-System above generally apply to NZCOM.
- There is no `DIR` command, you must use `SDZ` instead. If you don't like this, look into the `ALIAS` facility.
- For consistency with other ZCPR3 operating systems (ZPM3, Z3PLUS) the SHOW.COM and HELP.COM command files were renamed to ZSHOW.COM and ZHELP.COM

7.4 Digital Research CP/M 3

This is the Digital Research follow-up product to their very popular CP/M 2.2 operating system. While highly compatible with CP/M 2.2, it features many enhancements and is not 100% compatible. It makes direct use of banked memory to increase the user program space (TPA). It also has a new suite of support tools and help system.

Documentation

- CPM3 Users Guide
- CPM3 Command Summary
- CPM3 Programmers Guide
- CPM3 System Guide

Boot Disk

To create (or update) a CP/M 3 boot drive, you must place `CPMLDR.SYS` on the system track of the disk. You must also place `CPM3.SYS` and `CCP.COM` on the target drive as regular files. Do **not** place CPM3.SYS on the boot track. `CPMLDR.SYS` chain loads `CPM3.SYS` which must exist as a regular file on the disk. Subsequently, `CPM3.SYS` loads `CCP.COM`.

The CP/M 3 boot files are not included on the ROM disk due to space constraints. You will need to transfer the following files to your system from the RomWBW distribution directory Binary/CPM3. You can use XModem for this (or any of the mechanisms in Transferring Files.

- `CPMLDR.SYS`
- `CPM3.SYS` or `CPM3BNK.SYS`
- `CCP.COM`

The CPM3.SYS boot file is provided in 2 versions. In the Binary/CPM3 distribution directory, CPM3.SYS is the "non-banked" version of CP/M 3. The CPM3BNK.SYS file is the "banked" version of CP/M 3. You almost certainly want to transfer the banked CPM3BNK.SYS version.

After transferring the boot files to your RomWBW system, you will need to use SYSCOPY to place CPMLDR.SYS on the boot track of the target drive. CPM3.SYS and CCP.COM can be copied to the target drive using any standard file copy tool such as PIP or COPY.

You do not need to be booted into CP/M 3 to create or update a CP/M 3 disk. The recommended approach is to boot CP/M 2.2 or Z-System from ROM. Transfer the boot files to the RAM disk. Then simply copy the files onto the CP/M 3 disk. Assuming the target CP/M 3 disk is F:, you can use the following commands to place the files on the target drive:

```
SYSCOPY F:=A:CPMLDR.SYS
COPY A:CPM3BNK.SYS F:CPM3.SYS
COPY A:CCP.COM F:
```

Note in the example above that CPM3BNK.SYS is renamed to CPM3.SYS in the copy command.

Character Device Mapping

Character device mapping under CP/M 3 has 3 layers:

CP/M Logical Device –> CP/M Physical Device –> RomWBW HBIOS Device

The primary CP/M Logical Devices are:

Device	Description
CON:	Console input or output device
AUX:	Auxiliary Input or Output Device
LST:	List output device, usually the printer

There are various aliases for these devices. Please refer to the CP/M 3 Users Guide for more information.

The mapping of CP/M 3 Logical Devices to Physical Devices is performed using the DEVICE command.

CP/M 3 refers to Physical Character Devices using the COM device label. These COM devices are mapped directly to the RomWBW HBIOS Char devices as described below:

COM0: –> HBIOS Char 0
COM1: –> HBIOS Char 1

```
COM2: -> HBIOS Char 2
...
COMn: -> HBIOS Char n
```

Notes

- The `COPYSYS` command described in the DRI CP/M 3 documentation is not provided with RomWBW. The RomWBW `SYSCOPY` command is used instead.
- Although CP/M 3 is generally able to run CP/M 2.2 programs, this is not universally true. This is especially true of the utility programs included with the operating system. For example, the `SUBMIT` program of CP/M 3 is completely different/incompatible from the `SUBMIT` program of CP/M 2.2.
- RomWBW fully supports CP/M 3 file date/time stamping, but this requires that the disk be properly initialized for it. This process has not been performed on the CP/M 3 disk image. Follow the CP/M 3 documentation to complete this process, if desired.

7.5 Z3PLUS Z-System for CP/M-Plus

Z3PLUS is a much further refined version of Z-System (ZCPR 3.4). Z3PLUS was sold as an enhancement for existing users of CP/M 3. For this reason, (by design) Z3PLUS does not provide a way to boot directly from disk. Rather, it is loaded after the system boots into CP/M 3. A `PROFILE.SUB` file is included which automatically launches Z3PLUS as soon as CP/M 3 loads.

Z3PLUS is a companion product to NZ-COM, they are almost identical having been written by the same team. The only difference is the base operating system on which they run, but the architecture, the tools, libraries, files, etc are all primarily the same.

Z3PLUS is highly configurable. The RomWBW distribution has been configured in the most basic way possible. You should refer to the documentation to customize your system.

Z3PLUS has substantially more functionality than CP/M 3 or the basic Z-System. It is important to read the “Z3PLUS Users Manual.pdf” document in order to use this operating system effectively.

Documentation

- Z3PLUS Users Manual
- Z-System Users Guide
- ZCPR3.3 User Guide

Boot Disk

Since Z3PLUS boots via CP/M 3, you first must make the disk CP/M 3 bootable. This is not a simple process, as well as placing `CPMLDR.SYS` on the system track of the disk there are several files that are required on the disk itself.This is described in Digital Research CP/M 3 section above.

You will need to add a `PROFILE.SUB` file to auto-start Z3PLUS itself.

Character Device Mapping

Mapping of character devices to RomWBW HBIOS Character devices operates exactly the same as described in Digital Research CP/M 3. The CP/M 3 `DEVICE` command is used to manipulate the device mappings.

Notes

- All of the notes for Digital Research CP/M 3 above generally apply to Z3PLUS.
- Some applications in the Z3PLUS distribution have been upgraded with newer versions. This is done with in

7.6 ZPM3

Simeon Cran's ZPM3 is an interesting combination of the features of both CP/M 3 and ZCPR3. Essentially, it has the features of and compatibility with both.

Due to this dual compatibility, the ZPM3 distribution image contains most of the standard CP/M 3 files as well as a variety of common ZCPR3 applications. However, you will notice that user area 0 of the disk has only a few files. Most of the files are distributed among other user areas which is standard practice for ZCPR3. Most importantly, you will see most of the applications in user area 15. The applications can be executed from any user area because ZPM3 has a default search path that includes User 15.

The ZPM3 distribution comes with essentially no utility programs at all. In addition to the standard CP/M 3 utilities, RomWBW includes a variety of common ZCPR3 utilities.

Documentation

ZPM3 has no real documentation. You are expected to understand both CP/M 3 and ZCPR 3. The best source of information is:

- CPM3 Users Guide

- CPM3 Command Summary
- CPM3 Programmers Guide
- CPM3 System Guide
- Z-System Users Guide
- ZCPR3.3 User Guide

Boot Disk

To create (or update) a ZPM3 boot drive, you must place `ZPMLDR.SYS` on the system track of the disk. You must also place `CPM3.SYS`, `ZCCP.COM`, `ZINSTAL.ZPM`, and `STARTZPM.COM` on the target drive as regular files. Do **not** place CPM3.SYS on the boot track. `ZPMLDR.SYS` chain loads `CPM3.SYS` which must exist as a regular file on the disk. Subsequently, `CPM3.SYS` loads `CCP.COM`.

The CP/M 3 boot files are not included on the ROM disk due to space constraints. You will need to transfer the following files to your system from the RomWBW distribution directory Binary/ZPM3. You can use XModem for this (or any of the mechanisms in Transferring Files.

- `ZPMLDR.SYS`
- `CPM3.SYS`
- `ZCCP.COM`
- `ZINSTAL.ZPM`
- `STARTZPM.COM`

You may be surprised to see the file called `CPM3.SYS`. This is not a typo. Although it is called `CPM3.SYS`, it is ZPM and not the same as `CPM3.SYS` in the CPM3 directory. Also, unlike CP/M 3, ZPM3 is always banked, so you will not find two versions of the file. `CPM3.SYS` is a banked implementation of ZPM3.

After transferring the boot files to your RomWBW system, you will need to use `SYSCOPY` to place `ZPMLDR.SYS` on the boot track of the target drive. The remaining boot files can be copied to the target drive using any standard file copy tool such as `PIP` or `COPY`.

You do not need to be booted into ZPM3 to create or update a ZPM3 disk. The recommended approach is to boot CP/M 2.2 or Z-System from ROM. Transfer the boot files to the RAM disk. Then simply copy the files onto the ZPM disk. Assuming the target ZPM3 disk is F:, you can use the following commands to place the files on the target drive:

```
SYSCOPY F:=A:ZPMLDR.SYS
COPY A:CPM3.SYS F:CPM3.SYS
COPY A:CCP.COM F:
COPY A:ZINSTAL.ZPM F:
COPY A:STARTZPM.COM F:
```

Character Device Mapping

Mapping of character devices to RomWBW HBIOS Character devices operates exactly the same as described in Digital Research CP/M 3. The CP/M 3 `DEVICE` command is used to manipulate the device mappings.

Notes

- The ZPM3 operating system is contained in the file called CPM3.SYS which is confusing, but this is as intended by the ZPM3 distribution. I believe it was done this way to make it easier for users to transition from CP/M 3 to ZPM3.
- The command line editing key bindings were slightly modified by Jon Saxton when he applied a few ZPM3 fixes. This is explained in the file zpm3fix.txt in the distribution folder Source/ZPM3. The current key bindings are from the “New” column below.

Old	New	Function
^A	^A	one word left
^B	^B	to beginning/end of line
^C	^C	reboot if at start of line
^D	^D	right one char
^E	^E	get previous line
^F	^F	right one word
^G	^G	delete char at cursor
^H	^H	destructive backspace
^I	^I	(TAB) ignored here
^J	^J	(LF) exit editor
^K	^K	delete all to the right
^L	^L	ignored
^M	^M	(CR) exit editor
^N	^N	ignored
^O	^O	ignored
^P	^P	toggle printer echoing
^Q	^Q	toggle autoprompt (if enabled)
^R	^R	ignored
^S	^S	left one char
^T	^T	delete word at cursor
^U	^U	add line to history
^V	^V	clear line, delete from history
^W	^X	get next line from history

Old	New	Function
^X	^W	delete all to the left
^Y	^Y	clear line

7.7 QP/M

QP/M is another OS providing compatibility with and enhancements to CP/M 2.2. It is provided as a bootable disk image for RomWBW.

Refer to the ReadMe.txt file in Source/Images/d_qpm for more details regarding the RomWBW adaptation and customizations.

Documentation

- QP/M 2.7 Installation Guide and Supplements
- QP/M 2.7 Interface Guide
- QP/M 2.7 Features and Facilities

Boot Disk

To create or update a bootable QP/M Z-System disk, a special process is required. QP/M is not provided in source format. You are expected to install QP/M over an existing CP/M installation using the `QINSTALL.COM` application.

To update an existing QP/M boot disk with the latest RomWBW CBIOS, you must use 2 steps: apply the generic CP/M system track, then reinstall the QP/M components. To do this, you can perform the following steps:

1. Boot to the existing QP/M disk. At this point, drive A should be the QP/M disk that you wish to update. You may receive a warning about CBIOS/HBIOS version mismatch.
2. Use RomWBW `SYSCOPY` to place the stock RomWBW CP/M OS image onto the system tracks of the QP/M boot disk:

   ```
   SYSCOPY A:=x:CPM.SYS
   ```

   where x is the drive letter of your ROM Disk.
3. Run `QINSTALL` to overlay the QP/M OS components on your QP/M boot disk.

WARNING: `QINSTALL` has no mechanism for retaining previous non-default settings. Any previous non-default settings you previously made with `QINSTALL` will need to be reapplied. The pre-built RomWBW QP/M disk image includes a couple of specific non-default settings

to optimize use with RomWBW. Please review the notes in the ReadMe.txt file in Source/Images/d_qpm.

Character Device Mapping

Mapping of character devices to RomWBW HBIOS Character devices operates exactly the same as described in Digital Research CP/M 2.2. The mappings can be viewed or modified using the QP/M `QSTAT` command which is analogous to the CP/M 2.2 `STAT` command. Do **not** use the CP/M 2.2 `STAT` command under QP/M.

Notes

- QPM is not available as source. This implementation was based on the QPM binary distribution and has been minimally customized for RomWBW.
- When booted, the QPM startup banner will indicate CP/M 2.2. This is because QPM uses the CP/M 2.2 CBIOS code.
- QINSTALL is used to customize QPM. It is included on the disk image. You should review the notes in the ReadMe.txt file in Source/Images/d_qpm before making changes.
- In addition to the QPM disk image, all of the QPM distribution files can be found in the RomWBW distribution in the Source/Images/d_qpm/u0 directory.
- The QPM disk image is not included as one of the slices on the RomWBW Combo Disk Image. If you want to include QPM, you can do so by following the directions in Source/Images/Readme.txt.

7.8 UCSD p-System

This is a full implementation of the UCSD p-System IV.0 for Z80 running under RomWBW. Unlike the OSes above, p-System uses its own unique filesystem and is not interoperable with other OSes.

It was derived from the p-System Adaptable Z80 System. Unlike some other distributions, this implements a native p-System Z80 Extended BIOS, it does not rely on a CP/M BIOS layer.

The p-System is provided on a hard disk image file called psys.img. This must be copied to its own dedicated hard disk media (CF Card, SD Card, etc.). It is booted by selecting slice 0 of the corresponding hard disk unit at the RomWBW Boot Loader prompt. Do not attempt to use CP/M slices on the same disk.

Due to limitations in the p-System configuration mechanism, it does not recognize the arrow keys of an ANSI Terminal. To work around this, the following control keys have been defined:

Function	Key
Up	ctrl+E
Down	ctrl+X
Left	ctrl+S
Right	ctrl+D

Refer to Source/pSys/ReadMe.txt for more details about the p-System adaptation.

Documentation

- UCSD p-System Users Manual

Boot Disk

There is no mechanism provided to create a p-System boot disk from scratch under RomWBW. This has already been done as part of the porting process. You must use the provided p-System hard disk image file which is bootable.

Character Device Mapping

RomWBW Character Devices are automatically assigned to p-System devices at startup. The current HBIOS Console device is assigned to CONSOLE:.
The next available HBIOS Char device is assigned to REMIN:/REMOUT:. The next available HBIOS Char devices is assigned to PRINTER:

Notes

- There is no floppy support at this time.
- The hard disk image contains 6 p-System slices which are assigned to p-System unit numbers 4, 5, 9, 10, 11, and 12 which is standard for p-System. Slices 0-5 are assigned sequentially to these p-System unit numbers and it is not possible to reassign them. Unit #4 (slice 0) is bootable and contains all of the p-System distribution files. Unit #5 (slice 1) is just a blank p-System filesystem. The other units (9-12) have not been initialized, but this can be done from Filer using the Zero command.

- p-System relies heavily on the use of a full screen terminal. This implementation has been setup to expect an ANSI or DEC VT-100 terminal or emulator. The screen output will be garbled if no such terminal or emulator is used for console output.
- There is no built-in mechanism to move files in and out of p-System. However, the .vol files in Source/pSys can be read and modified by CiderPress. CiderPress is able to add and remove individual files.

  Andrew Davidson has created a Python script that can extract p-System volumes from an existing disk image file. The script is also capable of inserting a modified volume back into the disk image file. This tool is available at https://github.com/robosnacks/psysimg.

7.9 FreeRTOS

Phillip Stevens has ported FreeRTOS to run under RomWBW. FreeRTOS is not provided in the RomWBW distribution. FreeRTOS is available under the MIT licence and further general information is available at FreeRTOS.

Phillip may be contacted via his GitHub Page.

7.10 Fuzix

Fuzix is a Unix-ish operating system for small systems. It is the work of Alan Cox and is hosted on GitHub at https://github.com/EtchedPixels/FUZIX. Fuzix itself is a stand-alone operating system, but it frequently utilizes RomWBW to boot and launch on RomWBW-supported platforms.

For those Fuzix platforms that leverage RomWBW for startup, you will program your ROM with the normal RomWBW ROM – there is no Fuzix-specific ROM. A Fuzix disk image for your system is then written to your disk media. After booting your system via the normal RomWBW ROM, you start Fuzix simply by choosing the disk device containing the Fuzix image at the RomWBW Loader prompt.

To create a Fuzix disk image:

- Locate and download the Fuzix disk image for your system from https://www.fuzix.org/. For each platform, you will typically find two image files. An emulator image (`emu-xxx.img`) and a disk image (`disk.img`). You want the disk image file.
- Write the disk image file to your physical media (CF Card, SD Card, etc.) starting at the beginning of the media (first sector). Do not combine the Fuzix image with the

RomWBW disk images – they are entirely separate.

To boot into Fuzix:

- Insert your Fuzix disk media.
- Power-up or reset your system. RomWBW should load normally and bring you to the RomWBW Boot Loader prompt.
- Depending on the platform, Fuzix may be built to run at a different baud rate that the default RomWBW baud rate. If so, it is best to change your RomWBW baud rate prior to initiating the Fuzix startup. You can do this at the loader prompt with a command like this:

  ```
  I 0 38400
  ```

  Replace 38400 with the desired baud rate for Fuzix. You will be prompted to change your terminal's baud rate at this time.
- At the RomWBW Boot Loader prompt, enter the disk unit number of the Fuzix media. Fuzix should load and you will see device discovery/information messages that vary depending on your platform. This is a typical example:

  ```
  RCBus [RCZ180_nat_wbw] Boot Loader
  FP Switches = 0x00

  Boot [H=Help]: 2

  Booting Disk Unit 2, Slice 0, Sector 0x00000000...

  Volume "Fuzix 126 Loader" [0xF200-0xF400, entry @ 0xF200]...
  FUZIX version 0.4
  Copyright (c) 1988-2002 by H.F.Bower, D.Braun, S.Nitschke, H.Peraza
  Copyright (c) 1997-2001 by Arcady Schekochikhin, Adriano C. R. da Cunha
  Copyright (c) 2013-2015 Will Sowerbutts <wi...@sowerbutts.com>
  Copyright (c) 2014-2023 Alan Cox <al...@etchedpixels.co.uk>
  Devboot
  512kB total RAM, 448kB available to processes (15 processes max)
  Enabling interrupts ... ok.
  0000 : CF Card                                  - OK
  0001 :  - absent
  hda: hda1 hda2 (swap)
  bootdev:
  ```

- At the `bootdev:` prompt, enter `hda1`. Fuzix should load and you will be prompted for a date/time. Here is a typical example:

```
bootdev: hda1
Mounting root fs (root_dev=1, ro): OK
Starting /init
init version 0.9.1
Checking root file system.
Current date is Fri 2023-08-18
Enter new date:
Current time is 13:30:24
Enter new time:

 ^ ^
 n n   Fuzix 0.4
 >@<
       Welcome to Fuzix
 m m


login:
```

- At the `login:` prompt, enter `root`. No password is required. You should then get a Fuzix # command prompt.

```
login: root

Welcome to FUZIX.
#
```

You may now use Fuzix as desired. The general operation and use of Fuzix is outside of the scope of this document.

Chapter 8

Custom Applications

The operation of the RomWBW hosted operating systems is enhanced through several custom applications. You have already read about one of these – the `ASSIGN` command. These applications are functional on all of the OS variants included with RomWBW.

The applications discussed here are **not** the same as the built-in ROM applications mentioned previously. These applications run as commands within the operating systems provided by RomWBW. So, these commands are only available at an operating system prompt after an operating system has been loaded.

All of the RomWBW Custom Applications are built to function under all of the RomWBW Operating Systems (except for p-System). In general, the applications will automatically adapt as needed to the currently running operating system. One exception is `FDU` – the Floppy Disk Utility. This application requires that you pick the floppy disk interface you want to interact with.

There is more complete documentation of all of these applications in the related RomWBW manual "RomWBW Applications" found in the Doc directory of the distribution.

The following custom applications are found on the ROM disk and are, therefore, globally available.

Application	**Description
ASSIGN	Add, change, and delete drive letter assignments. Use ASSIGN /? for usage instructions.
SYSCOPY	Copy system image to a device to make it bootable. Use SYSCOPY with no parms for usage instructions.
MODE	Reconfigures serial ports dynamically.

Application	**Description
FDU	Format and test floppy disks. Menu driven interface.
FORMAT	Will someday be a command line tool to format floppy disks. Currently does nothing!
XM	XModem file transfer program adapted to hardware. Automatically uses primary serial port on system.
FLASH	Will Sowerbutts' in-situ ROM programming utility.
FDISK80	John Coffman's Z80 hard disk partitioning tool. See documentation in Doc directory.
FAT	Access MS-DOS FAT filesystems from RomWBW (based on FatFs).
TALK	Direct console I/O to a specified character device.
RTC	Manage and test the Real Time Clock hardware.
TIMER	Display value of running periodic system timer.
CPUSPD	Change the running CPU speed and wait states of the system.

Some custom applications do not fit on the ROM disk. They are found on the disk image files or the individual files can be found in the Binary/Apps directory of the distribution. They are also included on the floppy disk and hard disk images.

Application	Description
TUNE	Play .PT2, .PT3, .MYM audio files.
INTTEST	Test interrupt vector hooking.

Chapter 9

Real Time Clock & Date/Time Stamping

RomWBW supports a variety of real time clock hardware. If your system has this hardware, then it will be able to maintain the current date and time even while your system is turned off.

Additionally, depending on the operating system being used, you may be able to utilize date/time stamping of files. To facilitate this a CP/M clock driver (WBWCLK) has been included inside `CLOCKS.DAT` that will read the clock via a RomWBW HBIOS call

You can determine if your system has a real time clock present (and functioning) by looking at the boot messages. Here is an example of a boot message reflecting the detection of a valid real time clock module:

```
DSRTC: MODE=STD IO=0x8A Thu 2023-10-19 14:07:11 CHARGE=ON
```

This example is from a DSRTC clock module. You may have a different one, but it will always display the current date/time.

In some cases, your real time clock will support charging of the battery or super-capacitor while the system has power. The status of this charging is displayed.

If the date/time of your RTC needs to be updated, you will need to do this with one of the utilities described below. There is no ability to update the date/time of the RTC in the RomWBW Boot Loader or Monitor.

9.1 Date/Time Utilities

RomWBW includes two utilities for displaying or setting the date/time stored by the RTC. They are both a bit different and are briefly described below.

A third utility TESTCLOK is also included as part of ZSDOS

9.1.1 WDATE Utility

The WDATE utility (contributed by Kevin Boone) is an application that will display and/or update the current date/time. Its operation is described in RomWBW Applications. This utility works with any of the supported RomWBW RTC hardware. Here is an example of displaying and updating the date/time with this utility:

```
A>wdate
Thursday 19 October 14:14:43 2023

A>wdate 23 10 19 14 24 30

A>wdate
Thursday 19 October 14:24:34 2023
```

Note that WDATE does not have anything to do with date/time stamping of files. It merely displays and sets the real time clock value.

9.1.2 RTC Utility

Like WDATE, the RTC utility (contributed by Andrew Lynch) will let you display and set the current date/time. However, this utility only works with the DSRTC hardware (DS1302 chip). It is a "direct to hardware application". Its operation is described in RomWBW Applications. Here is an example of displaying and updating the date/time with this utility:

```
A>rtc
Start RTC Program
RomWBW HBIOS, Mark 4 RTC Latch Port 0x8A

RTC: Version 1.9
Commands: E)xit T)ime st(A)rt S)et R)aw L)oop C)harge N)ocharge D)elay I)nit G)et P)ut B)oot W
start H)elp

RTC>t
Current time: 23-10-19 14:30:25-05

RTC>i
Init date/time.
```

```
YEAR:23
MONTH:10
DATE:19
HOURS:14
MINUTES:31
SECONDS:00
DAY:05
```

The RTC utility is also capable of turning the charging feature of the DS1320 chip on or off. Here is an example of turning if off and back on:

```
A>rtc
Start RTC Program
RomWBW HBIOS, Mark 4 RTC Latch Port 0x8A


RTC: Version 1.9
Commands: E)xit T)ime st(A)rt S)et R)aw L)oop C)harge N)ocharge D)elay I)nit G)et P)ut B)oot W
start H)elp

RTC>n
Trickle charger disabled.

RTC>c
Trickle charger enabled.
```

Do **not** enable charging unless you are sure that your system supports this. If your RTC is being powered by a normal battery, it would be dangerous to enable charging.

9.1.3 TESTCLOK Utility

The TESTCLOK utility is used to test a selected CPM clock driver loaded from the CLOCKS.DAT file. After selecting the location of CLOCKS.DAT and the clock driver (45. WBWCLK) it displays the currently configured time until a key is pressed.

```
A>testclok

TESTCLOK V1.0    Copyright (C) 1988    H.F. Bower / C.W. Cotrill

Extract Clock from Library ([Y]/N) : Y
Location of CLOCKS.DAT [A0:] : <RETURN>
```

```
 1. ACTRIX            2. ALSPA             3. AMPRO-LB
 4. ANLYTCL-PRD       5. AP2-CDZ180        6. AP2-THND/MT
 7. AP2-TIMASTR       8. AP2E+PCP-TM       9. AP2E+PCPI
10. AP2E-THUNDR      11. AP2E-TMASTR      12. BIG-BD-II
13. BP-BIOS          14. CCS-WALLCLK      15. CPUPRO-SSB1
16. ELECTR-MFIO      17. EPSON-QX10       18. ETS180IO+
19. H19-SUPER19      20. H19-ULTRA        21. H19-WATZMAN
22. H89-BITZERO      23. H89-PC12         24. H89-WIDGET
25. H89-WISE         26. H89UTI           27. HEATH-BIOS
28. HOUSEMASTER      29. K83-HOLMES       30. KAYPRO-84
31. KENMOR-ZTIM      32. KPRO-ADVENT      33. KPRO-LEGACY
34. MD3-MACK         35. MTN100K-DAY      36. ONEAC-ON!
37. OTRANA-ATCH      38. P&T-HEARTBT      39. QTSYS-S100
40. RELATIVE         41. S100-5832        42. SB180-HRTBT
43. SB180-XBIOS      44. SIMHCLOK         45. WBWCLK
46. XEROX-820        47. ZSDOS-BIOS

Enter Clock Driver Selection : 45

..Loading : WBWCLK      ...
Linking Clock Module... OK
RomWBW HBIOS Clock      1.1

RomWBW Series HBIOS Clock

Press any key to quit...

 19 Oct 2023  14:24:34
```

Since this runs at the CPM driver level it is useful as an end-to-end test to prove that date time stamping is able to read the correct time

The `TESTCLOK` utility is provided by ZSDOS, please see the ZSDOS Manual for further information

9.2 Date/Time File Stamping

If an RTC is available in your system, then most operating systems can use it to date/time stamp files. This just means recording the date/time of file creation, update, and or access in

the directory. This capability is available in all of the RomWBW operating system except the original DRI CP/M 2.2.

Three types of date/time stamping are supported using realtime clock supported by RomWBW HBIOS. DateStamper, NZT and P2DOS.

In some cases (such as ZSDOS), you must load an RSX (memory resident utility) to enable date/time stamping of files. This could be automated using a `PROFILE.SUB` file. Preconfigured loaders are provided, bypassing the need to use SETUPZST.

Additionally, you will need to initialize the directory. The procedure varies depending on the date/time stamping mechanism, so you must review the associated documentation.

The date/time stamping mechanisms for each operating system are generally not compatible. If you initialize a directory for a type of stamping, you should be careful not to manipulate that directory with a different operating system with a different date/time stamping mechanism. Doing so may corrupt the directory.

The RomWBW disk images do not have date/time stamping initialized. This is to avoid any chance of directory corruption.

9.2.1 DateStamper

DateStamper datestamping follows the standard set by Plu*Perfect Systems. This method stores stamps in a disk file named `!!!TIME&.DAT`. Only DateStamper stamping stores full time and date stamps for file Creation, Last Modification, and Last Access, and may be used with any CP/M diskette format. In addition, the DateStamper protocol is supported by a mature set of compatible utilities.

Key Utilities

- LDDS.COM - Load DateStamper date/time stamping resident extension. (RomWBW Provided)
- PUTDS.COM - Prepare disk for DateStamper date/time stamping.

After using PUTDS to initialize a directory for ZDS date stamping, it may be necessary to run RELOG before the stamping routines will actually start working.

9.2.2 P2DOS (CP/M Plus compatible)

CP/M Plus-type datestamping is also widely used due to the popularity of CP/M Plus (also know as CP/M 3). CP/M Plus-type file datestamping uses directory sectors to store file datestamps which may be accessed more quickly by programs, but there is no Last File Access stamp.

Finally, the range of utilities for this type of stamps is more limited than for the DateStamper protocol.

Key Utilities

- LDP2D.COM - Load P2DOS date/time stamping resident extension. (RomWBW Provided)
- INITDIR.COM - Prepares disks for P2DOS-type file stamping.

9.2.3 NZT

The use of NZT needs to be further documented

Key Utilities

- LDNZT.COM - Load NZT date/time stamping resident extension. (RomWBW Provided)

9.2.4 Additional Notes

The following files have been provided, customised and tested for for use in RomWBW

- `CLOCKS.DAT` - Library of clock drivers, which has been updated to include the RomWBW clock driver WBWCLK, and also includes the SIMHCLOK clock driver. The file is just a standard LU type library and is easily updated using NULU. The members are the relocatable binaries, but with the .REL extension removed.
- `STAMPS.DAT` - Library of available date/time stamping modules for SETUPZST. The file has been replaced with an updated version from the Walnut Creek CP/M CDROM. The original version has a bug that prevents RSX (resident system extension) mode to load properly.

Additional Notes

- `SETUPZST` (provided by ZSDOS) Should not normally be needed since the creation of the appropriate LDTIM loaders has already been performed.
- `FILEDATE` only works with DateStamper style date stamping. If you run it on a drive that is not initialized for DateStamper, it will complain `FILEDATE, !!!TIME&.DAT missing`. This is normal and just means that you have not initialized that drive for DateStamper (using PUTDS).
- `ZXD` will handle either DateStamper or P2DOS type date stamping. However, it **must** be configured appropriately. As distributed, it will look for P2DOS date stamps. Use ZCNFG to reconfigure it for P2DOS if that is what you are using.

9.3 Timezone

None of the operating systems distributed with RomWBW have any concept of timezone. When files are date/time stamped, the date/time will simply be whatever date/time the RTC currently has.

The normal practice is to set the RTC to your local time. This implies that you would need to manually adjust the RTC for daylight savings time and/or when you travel to a different time zone.

The date/time stamps of files in directories will also be stored in local time. This includes files stored in a FAT filesystem. If you subsequently view the directory from modern machines (Windows, Linux, etc.), the date/time displayed will depend on the behavior of the modern system.

For example, Linux assumes that the date/time of files is UTC. So, if you create a file on a FAT filesystem with your RomWBW computer and then use Linux to view the directory, the date/time stamps will seem “off” by a few hours.

The only alternative you may consider is setting the date/time of your RTC to UTC. Since UTC is consistent across all timezones and daylight savings time, your file date/time stamps will also be consistent. Of course, this will mean that your RomWBW computer will display a date/time that seems wrong because it is not local time.

Chapter 10

CP/NET Networking

Digital Research created a simple network file sharing system called CP/NET. This allowed a network server running CP/NOS to host files available to network attached CP/M computers. Essentially, the host becomes a simple file sharing server.

RomWBW disk images include an adaptation of the DRI CP/NET client software provided by Douglas Miller. RomWBW does not support operation as a network server itself. However, Douglas has also developed a Java-based implementation of the DRI network server that can be used to provide host services from a modern computer.

Both CP/NET 1.2 and 3.0 clients are provided. Version 1.2 is for use with CP/M 2.2 and compatible OSes. Version 3.0 is for use with CP/M 3 and compatible OSes.

The CP/NET client software provided with RomWBW requires a supported ethernet interface module. At this time, the following are supported:

- RCBus MT011 w/ Ethernet Featherwing and (optionally) SPI FRAM (e.g., Adafruit SPI Non-Volatile FRAM Breakout)
- Duodyne Disk I/O w/ Wiz850IO and (optionally) SPI NVRAM (e.g., 25LC256)
- Generic Serial Interface

NOTE: The Generic Serial Interface is supported by RomWBW, but is not documented here. You must refer to the CP/NET documentation referenced below.

The client software interacts directly with this hardware. In a future version of RomWBW, I hope to add a generic networking API that will allow a greater range of network hardware to be used.

To use CP/NET effectively, you will want to review the documentation provided by Douglas on at his cpnet-z80 GitHub Project. Additionally, you should consult the DRI documentation

which is not included with RomWBW, but is available on the cpnet-z80 site.

Below, I will provide the general steps involved in setting up a network using MT011 with RomWBW.

10.1 CP/NET Client Setup

The CP/NET client files are included on the RomWBW disk images, but they are found in user area 4. They are placed there to avoid confusing anyone that is not specifically trying to run a network client. They are only found on the CPM 2.2 and CP/M 3 slices. Using CP/NET on alternative OSes may work, but is not officially supported.

The CP/NET client files are packaged in `.LBR` library files. The library files are found in user area 4.

File	CP/NET Version	OS	Hardware
CPN12MT.LBR	CP/NET 1.2	CP/M 2.2	RCBus w/ MT011
CPN3MT.LBR	CP/NET 3	CP/M 3	RCBus w/ MT011
CPN12DUO.LBR	CP/NET 1.2	CP/M 2.2	Duodyne w/ Disk I/O
CPN3DUO.LBR	CP/NET 3	CP/M 3	Duodyne w/ Disk I/O
CPN12SER.LBR	CP/NET 1.2	CP/M 2.2	RomWBW Serial Port
CPN3SER.LBR	CP/NET 3	CP/M 3	RomWBW Serial Port

First, you need to merge the files from the correct library file into user area 0. This is done by extracting the files using the `NULU` library management utility application.

1. Start NULU specifying desired CP/NET library for <filename>:

   ```
   A>NULU 4:<filename>
   ```

2. At the NULU prompt, extract the files using the `-E *.*` command:

   ```
   -READY A0:>-E *.*
   ```

3. Exit NULU using the `-X` command:

   ```
   -Extract members A0:>-x
   ```

Here is an example of extracting the CP/NET 1.2 client files for an RCBus system w/ MT011. You should be in user area 0 when performing this operation.

```
A>nulu 4:cpn12mt
NULU 1.52  (07/12/87)
```

```
Copyright (C) 1984, 1985 & 1987 by Martin Murray
Bug fixes in version 1.52 by Mick Waters

TYPE -H FOR HELP

Library A4:CPN12MT.LBR open.
(Buffer size: 259 sectors)
Active entries: 27, Deleted: 0, Free: 5, Total: 32.
-READY A0:>-e *.*
Extracting...
  CCP     .SPR to A0:CCP     .SPR
  CPM2NET .HLP to A0:CPM2NET .HLP
  CPNBOOT .COM to A0:CPNBOOT .COM
  CPNET12 .HLP to A0:CPNET12 .HLP
  CPNETLDR.COM to A0:CPNETLDR.COM
  CPNETSTS.COM to A0:CPNETSTS.COM
  DSKRESET.COM to A0:DSKRESET.COM
  ENDLIST .COM to A0:ENDLIST .COM
  LOCAL   .COM to A0:LOCAL   .COM
  LOGIN   .COM to A0:LOGIN   .COM
  LOGOFF  .COM to A0:LOGOFF  .COM
  MAIL    .COM to A0:MAIL    .COM
  NDOS    .SPR to A0:NDOS    .SPR
  NETDOWN .COM to A0:NETDOWN .COM
  NETSTAT .COM to A0:NETSTAT .COM
  NETWORK .COM to A0:NETWORK .COM
  NVRAM   .COM to A0:NVRAM   .COM
  PIPNET  .COM to A0:PIPNET  .COM
  RDATE   .COM to A0:RDATE   .COM
  SNIOS   .SPR to A0:SNIOS   .SPR
  SRVSTAT .COM to A0:SRVSTAT .COM
  TR      .COM to A0:TR      .COM
  WIZCFG  .COM to A0:WIZCFG  .COM
  WIZDBG  .COM to A0:WIZDBG  .COM
  WIZTEST .COM to A0:WIZTEST .COM
  XSUBNET .COM to A0:XSUBNET .COM
-Extract members A0:>-x

Closing A4:CPN12MT.LBR...
```

At this point, you will need to configure your ethernet adapter for your local network using `WIZCFG`. The definitive guide to the use of `WIZCFG` is on the cpnet-z80 site in the document called “CPNET-WIZ850io.pdf”. Here is an example of the commands needed to configure the WizNet:

`wizcfg w n F0`	set CP/NET node id
`wizcfg w i 192.168.1.201`	set WizNet IP address
`wizcfg w g 192.168.1.1`	set local network gateway IP address
`wizcfg w s 255.255.255.0`	set WizNet subnet mask
`wizcfg w 0 00 192.168.1.3 31100`	set server node ID, IP address, & port

You will need to use values appropriate for your local network. You can use the command `wiznet w` to display the current values which is useful to confirm they have been set as intended.

```
A>wizcfg w
Node ID:  F0H
IP Addr:  192.168.1.201
Gateway:  192.168.1.1
Subnet:   255.255.255.0
MAC:      98:76:B6:11:00:C4
Socket 0: 00H 192.168.1.3 31100 0
```

These values can be persisted across power-cycles if your system has NVRAM storage. To program the values into your NVRAM, you would use the same commands as above, but omit the `w` parameter. The “CPNET-WIZ850io.pdf” document is highly recommended to understand the operation of `WIZCFG`.

If you do not utilize NVRAM to persist your configuration, you will need to reapply these commands every time you power cycle your RomWBW computer, so I recommend putting them into a `SUBMIT` file.

After applying these commands, you should be able ping the WizNet from another computer on the local network. If this works, then the client-side is ready.

10.2 CP/NET Sever Setup

These instructions will assume you are using Douglas Miller’s CpnetSocketServer to implement a CP/NOS server on your network. The definitive guide to this software is also on the [cpnet-z80] (https://github.com/durgadas311/cpnet-z80) site and is called “CpnetSocketServer.pdf”.

The software is a Java application, so it can generally run anywhere there is a Java runtime environment available. I have normally used it on a Linux system and have had good results with that.

You will need to download the application called “CpnetSocketServer.jar” from the cpnet-z80 site. The application uses a configuration file. My configuration file is called “cpnet00.rc” and has these contents:

```
cpnetserver_host = 192.168.1.3
cpnetserver_port = 31100
cpnetserver_temp = P
cpnetserver_sid = 00
cpnetserver_max = 16
cpnetserver_root_dir = /home/wayne/cpnet/root
```

You will also need to setup a directory structure with the drive letters per the documentation.

To start the server, you would use a command like this:

```
java -jar CpnetSocketServer.jar conf=cpnet00.rc
```

At this point, the server should start and you should see the following:

```
CpnetSocketServer v1.3
Using config in cpnet00.rc
Server 00 Listening on 192.168.1.3 port 31100 debug false
```

Your CP/NET server should now be ready to accept client connections.

10.3 CP/NET Usage

With both the client and server configured, you are ready to load and use CP/NET on your RomWBW system. CP/NET documentation is available on the cpnet-z80 site. The document is called “dri-cpnet.pdf”.

Under CP/M 2.2, you will start the networking client using the command `CPNETLDR`. Under CP/M 3, you use the command `NDOS3`. If that works, you can map network drives as local drives using the `NETWORK` command. The `CPNETSTS` command is useful for displaying the current status. Here is a sample session from CP/M 2.2:

```
A>cpnetldr


CP/NET 1.2 Loader
```

```
=================

BIOS         E600H  1A00H
BDOS         D800H  0E00H
SNIOS   SPR  D400H  0400H
NDOS    SPR  C800H  0C00H
TPA          0000H  C800H

CP/NET 1.2 loading complete.

A>network k:=c:[0]

A>dir k:
K: TELNET   COM : ZDENST   COM : CLRDIR   COM : RTC      COM
K: DDTZ     COM : MBASIC   COM : XSUBNET  COM : NETWORK  COM
K: WGET     COM : UNCR     COM : FLASH    COM : PIP      COM
K: TIMEZONE COM : COMPARE  COM : ZAP      COM

A>cpnetsts

CP/NET 1.2 Status
=================
Requester ID = F0H
Network Status Byte = 10H
Disk device status:
  Drive A: = LOCAL
  Drive B: = LOCAL
  Drive C: = LOCAL
  Drive D: = LOCAL
  Drive E: = LOCAL
  Drive F: = LOCAL
  Drive G: = LOCAL
  Drive H: = LOCAL
  Drive I: = LOCAL
  Drive J: = LOCAL
  Drive K: = Drive C: on Network Server ID = 00H
  Drive L: = LOCAL
  Drive M: = LOCAL
  Drive N: = LOCAL
  Drive O: = LOCAL
```

```
  Drive P: = LOCAL
Console Device = LOCAL
List Device = LOCAL
```

If you are using CpSocketServer to provide the CP/NOS server, then you will see some messages on your server console when clients connect. Here are the messages issued by the server in the above example:

```
Connection from 192.168.1.201 (31100)
Remote 192.168.1.201 is f0
Creating HostFileBdos 00 device with root dir /home/wayne/cpnet/root
```

At this point CP/NET is ready for general use. You should be able to access files on the network mapped drives just like files on your local drives.

10.4 Network Boot

It is possible to boot your RomWBW system directly from a network server if it has the required hardware. This means that the operating system will be loaded directly from the network server and all of your drive letters will be provided by the network server. The supported hardware is:

- RCBus System w/ MT011 including:
  - Featherwing WizNet W5500
  - SPI FRAM on secondary SPI interface (CS2)
- Duodyne Disk I/O Board including:
  - WIZ850io Module
  - 25LCxxx Serial SPI EEPROM

Unlike the CP/NET Client, the presence of dedicated non-volatile storage is required to hold the network configuration. This will be FRAM (for MT011) or Serial SPI EEPROM (Duodyne). The NVRAM is used to store your WizNet configuration values so they do not need to be re-entered every time you power-cycle your system.

It is important to understand that the operating system that is loaded in this case is **not** a RomWBW enhanced operating system. Some commands (such as the `ASSIGN` command) will not be possible. Also, you will only have access to drives provided by the network server – no local disk drives will be available.

Using the same values from the previous example, you would issue the `WIZCFG` commands:

```
wizcfg n F0
wizcfg i 192.168.1.201
wizcfg g 192.168.1.1
```

```
wizcfg s 255.255.255.0
wizcfg 0 00 192.168.1.3 31100
```

Note that the 'w' parameter is now omitted which causes these values to be written to NVRAM.

As before, your network server will need to be running CpnetSocketServer. However, you will need to setup a directory that contains some files that will be sent to your RomWBW system when the Network boot is performed. By default the directory will be `~/NetBoot`. In this directory you need to place the following files:

- `cpnos.sys`
- `ndos.spr`
- `snios.spr`

You also need to make sure CpnetSocketServer is configured with an 'A' drive and that drive must contain (at an absolute minimum) the following file:

- `ccp.spr`

All of these files are found in the Binary/CPNET/NetBoot directory of the RomWBW distribution. You will find 2 sub-directories named MT and DUO. Get the files from the sub-directory corresponding to your specific hardware.

Finally, you need to add the following line to your CpnetSocketServer configuration file:

```
netboot_default = cpnos.sys
```

To perform the network boot, you start your RomWBW system normally which should leave you at the Boot Loader prompt. The 'N' command will initiate the network boot. Here is an example of what this looks like:

```
RCBus [RCZ180_nat_wbw] Boot Loader

Boot [H=Help]: n

Loading Network Boot...
MT011 WizNET Network Boot

WBWBIOS  SPR  FD00 0100
COBDOS   SPR  FA00 0300
SNIOS    SPR  F600 0400
NDOS     SPR  EA00 0C00

58K TPA
```

```
A>
```

The CP/M operating system and the CP/NET components have been loaded directly from the network server. All of your drive letters are automatically mapped directly to the drive letters configured with CpnetSocketServer.

```
A>cpnetsts

CP/NET 1.2 Status
=================
Requester ID = F0H
Network Status Byte = 10H
Disk device status:
  Drive A: = Drive A: on Network Server ID = 00H
  Drive B: = Drive B: on Network Server ID = 00H
  Drive C: = Drive C: on Network Server ID = 00H
  Drive D: = Drive D: on Network Server ID = 00H
  Drive E: = Drive E: on Network Server ID = 00H
  Drive F: = Drive F: on Network Server ID = 00H
  Drive G: = Drive G: on Network Server ID = 00H
  Drive H: = Drive H: on Network Server ID = 00H
  Drive I: = Drive I: on Network Server ID = 00H
  Drive J: = Drive J: on Network Server ID = 00H
  Drive K: = Drive K: on Network Server ID = 00H
  Drive L: = Drive L: on Network Server ID = 00H
  Drive M: = Drive M: on Network Server ID = 00H
  Drive N: = Drive N: on Network Server ID = 00H
  Drive O: = Drive O: on Network Server ID = 00H
  Drive P: = Drive P: on Network Server ID = 00H
Console Device = LOCAL
List Device = LOCAL
```

At this point you can use CP/M and CP/NET normally, but all disk access will be to/from the network drives. There is no access to your local disk drives in this boot mode.

Chapter 11

Transferring Files

Transferring files between your modern computer and your RomWBW system can be achieved in a variety of ways. The most common of these are described below. All of these have a certain degree of complexity and I encourage new users to use the available community forums to seek assistance as needed.

11.1 Serial Port Transfers

RomWBW provides an serial file transfer program called XModem that has been adapted to run under RomWBW hardware. The program is called `XM` and is on your ROM disk as well as all of the pre-built disk images.

You can type `XM` by itself to get usage information. In general, you will run `XM` with parameters to indicate you want to send or receive a file on your RomWBW system. Then, you will use your modern computers terminal program to complete the process.

The `XM` application generally tries to detect the hardware you are using and adapt to it. However, you must ensure that you have a reliable serial connection. You must also ensure that the speed of the connection is not too fast for XModem to service. If your file transfer attempts are failing, try either of the following:

- Check that hardware flow control is enabled in your terminal emulation software.
- Reduce the speed of your serial port connection.

There is an odd interaction between XModem and partner terminal programs that can occur. Essentially, after launching `XM`, you must start the protocol on your modern computer fairly quickly (usually in about 20 seconds or so). So, if you do not pick a file on your modern

computer quickly enough, you will find that the transfer completes about 16K, then hangs. The interaction that causes this is beyond the scope of this document.

11.2 Disk Image Transfers

It is possible to pass disk images between your RomWBW system and your modern computer. This assumes you have an appropriate media slot on your modern computer for the media you want to use (CF Card, SD Card, floppy drive, etc.).

The general process to get files from your modern computer to a RomWBW computer is:

1. Use `cpmtools` on your modern computer to create a RomWBW CP/M filesystem image.
2. Insert your RomWBW media (CF Card, SD Card, floppy disk, etc.) in your modern computer.
3. Use a disk imaging tool to copy the RomWBW filesystem image onto the media.
4. Move the media back to the RomWBW computer.

This process is a little complicated, but it has the benefit of allowing you to get a lot of files over to your RomWBW system quickly and with little chance of corruption.

The process can be run in reverse to get files from your RomWBW computer to a modern computer.

The exact use of these tools is a bit too much for this document, but the tools are all included in the RomWBW distribution along with usage documents.

Note that the build scripts for RomWBW create the default disk images supplied with RomWBW. It is relatively easy to customize the contents of the disk images that are part of RomWBW. This is described in more detail in the Source/Images directory of the distribution.

11.3 FAT Filesystem Transfers

The FAT filesystem format that originated with MS-DOS is almost ubiquitous across modern computers. Virtually all operating systems now support reading and writing files to a FAT filesystem. For this reason, RomWBW now has the ability to read and write files on FAT filesystems.

This capability means that you can generally use your modern computer to make an SD Card, CF Card, or USB Drive with a standard FAT filesystem on it, then place that media in your RomWBW computer and access the files.

- Files can be copied between a FAT filesystem and a CP/M filesystem, but you cannot execute files directly from a FAT filesystem.
- FAT12, FAT16, and FAT32 formats are supported.
- Long filenames are not supported. Files with long filenames will show up with their names truncated into the older 8.3 convention. If you have files on your modern computer with long filenames, it is usually easiest to rename them on the modern computer.
- A FAT filesystem can be located on floppy or hard disk media. For hard disk media, a valid FAT Filesystem partition must exist.

Some additional **Notes** and **WARNINGS** Things to be careful about

- CP/M (and compatible) OSes do not support all of the filename characters that a modern computer does. The following characters are **not permitted** in a CP/M filename:

  ```
  < > . , ; : = ? * [ ] _ % | ( ) / \
  ```

  The FAT application does not auto-rename files when it encounters invalid filenames. It will just issue an error and quit. Additionally, the error message is not very clear about the problem.
- Microsoft Windows will sometimes suggest **reformatting** partitions that it does not recognize (e.g. RomWBW). If you are using media that contains both a FAT partition and a RomWBW partition you may prompted to format a partition of your SD/CF/USB Media when inserting the card into a Windows computer, you probably want to select Cancel.

11.3.1 FAT Filesystem Preparation

In general, you can create media formatted with a FAT filesystem on your RomWBW computer or on your modern computer. We will only be discussing the RomWBW-based approach here.

Floppy Disk

In the case of a floppy disk, you can use the `FAT` application to format the floppy disk. The floppy disk must already be physically formatted using RomWBW FDU or equivalent. If your floppy disk is on RomWBW disk unit 2, you could use `FAT FORMAT 2:`. This will overwrite the floppy with a FAT filesystem and all previous contents will be lost. Once formatted this way, the floppy disk can be used in a floppy drive attached to a modern computer or it can be used on RomWBW using the other `FAT` tool commands.

Hard Disk

In the case of hard disk media, it is necessary to have a FAT partition. If you prepared your RomWBW hard disk media using the disk image process, then this partition will already be defined and you do not need to recreate it. This default FAT partition is located at approximately 512MB from the start of your disk and it is 384MB in size. So, your hard disk media must be 1GB or greater to use this default FAT partition.

You can confirm the existence of the FAT partition with `FDISK80` by using the 'P' command to show the current partition table. Here is an example of a partition table listing from `FDISK80` that includes the FAT partition (labelled "FAT16"):

```
Capacity of disk 4:  (  4G)  7813120      Geom 77381010
Nr  ---Type- A --      Start         End   LBA start  LBA count  Size
 1    RomWBW   2e      8:0:1  1023:15:16        2048    1048576  512M
 2     FAT16   06   1023:0:1  1023:15:16     1050624     786432  384M
 3             00       *** empty ***
 4             00       *** empty ***
```

If your hard disk media does not have a FAT partition already defined, you will need to define one using FDISK80 by using the 'N' command. Ensure that the location and size of the FAT partition does not overlap any of the CP/M slice area and that it fits within the size of your media.

Once the partition is defined, you will still need to format it. Just as with a floppy disk, you use the `FAT` tool to do this. If your hard disk media is on RomWBW disk unit 4, you would use `FAT FORMAT 4:`. This will look something like this:

```
E>fat format 4:

About to format FAT Filesystem on Disk Unit #4.
All existing FAT partition data will be destroyed!!!

Continue (y/n)?

Formatting... Done
```

Your FAT filesystem is now ready to use.

If your RomWBW system has multiple disk drives/slots, you can also just create a disk with your modern computer that is a dedicated FAT filesystem disk. You can use your modern computer to format the disk (floppy, CF Card, SD Card, etc.), then insert the disk in your RomWBW computer and access it using `FAT` based on its RomWBW unit number.

11.3.2 FAT Application Usage

Transferring files is accomplished by running a RomWBW custom application called `FAT`. This application understands both FAT filesystems as well as CP/M filesystems.

To copy files from your modern computer to your RomWBW computer, start by putting the disk media with the FAT filesystem in your modern computer. The modern computer should recognize it. Then copy the files you want to get to your RomWBW computer onto this media. Once done, remove the media from your modern computer and insert it in the RomWBW computer.

Finally, use the `FAT` tool to copy the files onto a CP/M drive. This process works just fine in reverse if you want to copy files from a CP/M filesystem to your modern computer.

Complete instructions for the `FAT` application are found in RomWBW Applications. Here, we will just provide a couple of simple examples. Note that the FAT application is not on the ROM disk because it is too large to include there.

The most important thing to understand about the `FAT` application is how it refers to FAT filesystems vs. CP/M filesystems. It infers this based on the file specification provided. If you use a specification like `C:SAMPLE.TXT`, it will use the C: drive of your CP/M operating system. If you use a specification like `4:SAMPLE.TXT`, it will use the FAT filesystem on the disk in RomWBW disk unit 4. Basically, if you start your file or directory specification with a number followed by a colon, it means FAT filesystem. Anything else will mean CP/M filesystem.

Here are a few examples. This first example shows how to get a FAT directory listing from RomWBW disk unit 4:

```
E>fat dir 4:

Directory of 4:


E>
```

As you can see, there are currently no files there. Now let's copy a file from CP/M to the FAT directory:

```
E>fat copy sample.txt 4:

Copying...

SAMPLE.TXT ==> 4:/SAMPLE.TXT ... [OK]
```

```
    1 File(s) Copied
```

If we list the FAT directory again, you will see the file:

```
E>fat dir 4:

Directory of 4:

01/30/2023  17:50:14         29952  ---A  SAMPLE.TXT
```

Now let's copy the file from the FAT filesystem back to CP/M. This time we will get a warning about overwriting the file. For this example, we don't want to do that, so we abort and reissue the command specifying a new filename to use:

```
E>fat copy 4:sample.txt e:

Copying...

4:/SAMPLE.TXT ==> E:SAMPLE.TXT Overwrite? (Y/N) [Skipped]

    0 File(s) Copied

E>fat copy 4:sample.txt e:sample2.txt

Copying...

4:/SAMPLE.TXT ==> E:SAMPLE2.TXT ... [OK]

    1 File(s) Copied
```

Finally, let's try using wildcards:

```
E>fat copy sample*.* 4:

Copying...

SAMPLE.TXT ==> 4:/SAMPLE.TXT Overwrite? (Y/N) ... [OK]
SAMPLE2.TXT ==> 4:/SAMPLE2.TXT ... [OK]

    2 File(s) Copied
```

Chapter 12

Customizing RomWBW

12.1 Startup Command Processing

Most of the operating systems supported by RomWBW provide a mechanism to run commands at boot. This is similar to the AUTOEXEC.BAT files from MS-DOS.

With the exception of ZPM3 and p-System, all operating systems will look for a file called `PROFILE.SUB` on the system drive at boot. If it is found, it will be processed as a standard CP/M submit file. You can read about the use of the SUBMIT facility in the CP/M manuals included in the RomWBW distribution. Note that the boot disk must also have a copy of `SUBMIT.EXE`.

Note that the automatic startup processing generally requires booting to a disk drive. Since the ROM disk is not writable, there is no simple way to add/edit a `PROFILE.SUB` file there. If you want to customize your ROM and add a `PROFILE.SUB` file to the ROM Disk, it will work, but is a lot harder than using a boot disk.

In the case of ZPM3, the file called `STARTZPM.COM` will be run at boot. To customize this file, you use the ZCPR ALIAS facility. You will need to refer to ZCPR documentation for more information on the ALIAS facility.

p-System has its own startup command processing mechanism that is covered in the p-System documentation.

12.2 NVRAM Configuration

See section Setting NVRAM Options for information about how to apply NVRAM configuration.

12.3 ROM Customization

The pre-built ROM images are configured for the basic capabilities of each platform. Additionally, some of the typical add-on hardware for each platform will be automatically detected and used. If you want to go beyond this, RomWBW provides a very flexible configuration mechanism based on configuration files. Creating a customized ROM requires running a build script, but it is quite easy to do.

Essentially, the creation of a custom ROM is accomplished by updating a small configuration file, then running a script to compile the software and generate the custom ROM and disk images. There are build scripts for Windows, Linux, and MacOS to accommodate virtually all users. All required build tools (compilers, assemblers, etc.) are included in the distribution, so it is not necessary to setup a build environment on your computer.

RomWBW can be built on modern Windows, Linux, or MacOS computers. The process for building a custom ROM is documented in the ReadMe.txt file in the Source directory of the distribution. Any modern version of Windows (32-bit or 64-bit), MacOS, or Linux released in the last 10 years should be able to run the build process.

For those who are interested in more than basic system customization, note that all source code is provided (including the operating systems). Modification of the source code is considered an expert level task and is left to the reader to pursue.

Note that the ROM customization process does not apply to UNA. All UNA customization is performed within the ROM setup script that is built into the ROM.

12.4 ROM User Application

The User App is provided as a way to access a custom written ROM application. In the pre-built ROMs, selecting User App will just return to the Boot Loader menu. If you are interested in creating a custom application to run instead, review the “usrrom.asm” file in the Source/HBIOS folder of the distribution.

Chapter 13

Upgrading

Upgrading to a newer release of RomWBW is essentially just a matter of updating the ROM chip in your system. If you have spare ROM chips for your system and a ROM programmer, it is always safest to retain your existing, working ROM chip and program a new one with the new firmware. If the new one fails to boot, you can easily return to the known working ROM.

For each ROM that comes with the RomWBW distribution, you will find that there are actually 3 different variants:

- `.rom` contains the complete ROM chip image
- `.upd` contains the system code, but omits the ROM Disk contents
- `.com` contains a CP/M executable version of the ROM code

So, for example, RCZ80_std contains the following files:

- `SBC_std.rom`
- `SBC_std.upd`
- `SBC_std.com`

The use of the `.com` variant is described below in Application Boot.

As previously discussed, the ROM in most RomWBW systems contains both the system code as well as a ROM Disk with files on it. The `.rom` variant of the ROM contains a full ROM chip image including both the system code and the Rom Disk contents. The `.upd` variant of the ROM contains only the system code portion of the ROM. If you apply the `.upd` variant to your system, it will overlay the system code, but will not overlay the ROM Disk contents (they remain intact). You may use either the `.rom` or the `.upd` file when updating your ROM chip (this does not apply to Application Boot). It is best to use the `.rom` file for your upgrade because the files on your ROM Disk should be updated whenever you update your system code. The

advantage of the `.upd` variant is that is much smaller, so you can upload and apply it faster. The ROM update instructions below generally refer to using the `.rom` variant. However, you may substitute the `.upd` variant if desired.

13.1 Application Boot

Prior to attempting to reprogram your actual ROM chip, you may wish to "try" the update to ensure it will work on your system. With RomWBW, you can upload a new ROM image executable and load it from the command line. For each ROM image file (.rom) in the Binary directory, you will find a corresponding application file (.com). For example, for SBC_std.rom, there is also an SBC_std.com file.

You can upload the `.com` file to your system using XModem, then simply run the `.com` file. You will see your system go through the normal startup process just like it was started from ROM. However, your physical ROM has not been updated and the next time you boot your system, it will revert to the system image contained in ROM.

Using this `.com` version of a ROM is an excellent way to confirm that the new ROM code you intend to program will work correctly. If it does not, then you can just reboot and your old ROM will be loaded.

When the `.com` file is loaded, you will be taken to the normal Boot Loader menu. However, you will find that the only OS that is available to boot from ROM is ZSDOS. There is only room for a single OS in the .com file. Even if you don't normally use ZSDOS, this will still confirm that your system operates well under the new ROM code.

13.2 Upgrading via Flash Utility

If you do not have easy access to a ROM programmer, it is usually possible to reprogram your system ROM using the FLASH utility from Will Sowerbutts. This application, called FLASH.COM, can be found on the ROM drive of any running system. In this case, you would need to transfer the new ROM image (.rom) over to your system using XModem (or one of the other mechanisms described in the Transferring Files section). The ROM image is too large to fit on your RAM drive, so you will need to transfer it to a larger storage drive. Once the ROM image is on your system, you can use the FLASH application to update your ROM. The following is a typical example of transferring ROM image using XModem and flashing the chip in-situ.

NOTE: The FLASH utility **can not** determine the type of your ROM chip if it is write protected. Additionally, it has no way to determine if it is write protected. If the FLASH utility indicates it does not recognize your ROM chip, check to ensure the chip is not write protected.

```
E>xm r rom.rom

XMODEM v12.5 - 07/13/86
RBC, 28-Aug-2019 [WBW], ASCI

Receiving: E0:ROM.IMG
7312k available for uploads
File open - ready to receive
To cancel: Ctrl-X, pause, Ctrl-X

Thanks for the upload

E>flash write rom.rom
FLASH4 by Will Sowerbutts <will@sowerbutts.com> version 1.2.3

Using RomWBW (v2.6+) bank switching.
Flash memory chip ID is 0xBFB7: 39F040
Flash memory has 128 sectors of 4096 bytes, total 512KB
Write complete: Reprogrammed 2/128 sectors.
Verify (128 sectors) complete: OK!
```

Obviously, there is some risk to this approach since any issues with the programming or ROM image could result in a non-functional system.

To confirm your ROM chip has been successfully updated, restart your system and boot an operating system from ROM. Do not boot from a disk device yet. Review the boot messages to see if any issues have occurred.

13.3 Upgrading via XModem Flash Updater

Similar to using the Flash utility, the system ROM can be updated or upgraded through the ROM-based updater utility. This works by reprogramming the flash ROM as the file is being transferred.

Using the ROM-based updater has the advantage that secondary storage is not required to hold the new image. In other words, it is not necessary to have a mass storage device available to store the ROM image.

From the Boot Loader menu select X (Xmodem Flash Updater) and then U (Begin Update). Then initiate the Xmodem transfer of the `.rom` file. Since the XModem Flash Updater will be relatively slow, you may wish to use the `.upd` variant of the ROM.

More information can be found in the ROM Applications document.

13.4 Post Upgrade Update

Once you are satisfied that the ROM is working well, you will need to update the system images and RomWBW custom applications on your disk drives. The system images and custom applications are matched to the RomWBW ROM firmware in use. If you attempt to boot a disk or run applications that have not been updated to match the current ROM firmware, you are likely to have odd problems.

The simplest way to update your disk media is to just use your modern computer to overwrite the entire media with the latest disk image of your choice. This process is described below in the Disk Images section. If you wish to update existing disk media in your system, you need to perform the following steps.

If the disk is bootable, you need to update the system image on the disk using the procedure described in the Operating Systems section of this document.

Finally, if you have copies of any of the RomWBW custom applications on your hard disk, you need to update them with the latest copies. The following applications are found on your ROM disk. Use COPY to copy them over any older versions of the app on your disk:

- ASSIGN.COM
- SYSCOPY.COM
- MODE.COM
- FDU.COM
- FORMAT.COM
- XM.COM
- FLASH.COM
- FDISK80.COM
- TALK.COM
- RTC.COM
- TIMER.COM
- FAT.COM

For example: `B>COPY ASSIGN.COM C:`

Some RomWBW custom applications are too large to fit on the ROM disk. If you are using any of these you will need to transfer them to your system and then update all copies. These applications are found in the Binary/Apps directory of the distribution and in all of the disk images.

- TUNE.COM

The files normally contained on the standard ROM Disk is based on a 512K ROM. If your system has a smaller size ROM, then not all of these files will be included on your ROM Disk. You will need to copy them to your system from the /Binary/Apps folder of the RomWBW distribution.

WARNING: If you run a RomWBW-specific application that is not the appropriate for the version of RomWBW you are running, the application will generate an error message and abort.

13.5 System Update

As previously described, a RomWBW ROM contains ROM applications as well as a ROM disk image. If you are upgrading your ROM with a new patch level release, you may wish to upgrade just the application portion of the ROM. This is referred to as a System Update.

If the system running ROMWBW utilizes the SST39SF040 Flash chip then it is possible to do a System Update in place of a System Upgrade in some cases. A System Update would involve only updating the BIOS, ROM applications and ROM-hosted operating systems.

A System Update may be more favorable than a System Upgrade in cases such as:

- Overwriting of the ROM drive contents is not desired.
- Temporary disk space is unavailable to hold a full ROM image.
- To reduce the time taken to transfer and flash a full ROM.
- Configuration changes are only minor and do not impact disk applications.

The RomWBW build process generates a system update file along with the normal ROM image and can be identified by the extension ".upd". It will be 128Kb in size. In comparison the normal ROM image will have the extension ".rom" and be 512Kb or 1024Kb in size.

Transferring and flashing the System Update is accomplished in the same manner as described above in Upgrading via Flash Utility with the required difference being that the flash application needs to be directed to complete a partial flash using the /P command line switch.

```
E>FLASH WRITE ROM.UPD /P
```